

National Taiwan University

Tideology

Wayne Wei, Duda Liao, Ian Tsai

- 1 Contest
- 2 Mathematics
- 3 Data structures
- 4 Numerical
- 5 Number theory
- 6 Combinatorial
- 7 Graph
- 8 Geometry
- 9 Strings
- 10 Various

Contest (1)

todo.txt	4 lines
<pre>code run chmod +x run code tem.cpp ./run tem</pre>	
template.cpp	45 lines
<pre>#include <bits/stdc++.h> #include <bits/extc++.h> using namespace std; #define int long long #define ll long long #define all(x) x.begin(), x.end() #define pb push_back #define F first #define S second #define pii pair<int, int> #define chmax(x, y) x = x > y ? x : y #define chmin(x, y) x = x < y ? x : y #define rep(i, a, b) for(int i = a; i < (b); ++i) #define sz(x) (int)(x).size() #define vi vector<int> #ifdef tsao template <typename T> ostream& operator << (ostream &o, const vector<T> &v){ o << "{"; int f = 0; for(auto &i : v) o << (f++ ? " " : "") << i; return o << "}"; } void bug__(int c, auto ...a){ cerr << "\e[1;" << c << "m"; (... , (cerr << a << " ")); cerr << "\e[0m\n"; } #define bug_(c, x...) bug__(c, __LINE__, "[" + string(#x) + "]" , x) #define bug(x...) bug_(32, x) #define bugv(x...) bug_(36, vector(x))</pre>	

1	<pre>#define safe bug_(33, "safe") #else #define bug(x...) void(0) #define bugv(x...) void(0) #define safe void(0) #endif 3 void solve(){ 5 } signed main(){ ios::sync_with_stdio(0); cin.tie(0);cout.tie(0); int t = 1; while(t--) solve(); 11 }</pre>
12	fast.cpp
19	30 lines
24	<pre>#pragma GCC optimize("Ofast,no-stack-protector") #pragma GCC optimize("no-math-errno,unroll-loops") #pragma GCC target("sse,sse2,sse3,ssse3,sse4") #pragma GCC target("popcnt,abm,mmx,avx,arch=skylake") __builtin_ia32_ldmxcsr(__builtin_ia32_stmxcsr() 0x8040) #include<unistd.h> char IB[1 << 16], OB[1 << 16]; int IP, IL, OP; inline int RC(){ if(IP == IL) IP = 0, IL = fread(IB, 1, sizeof IB, stdin); return IP == IL ? -1 : IB[IP++]; } inline int R(){ int c = RC(), s = 1; unsigned long long x = 0; while(c != -1 and c != '-' and (c < '0' or c > '9')) c = RC() ; if(c == '-') s = -1, c = RC(); while(c >= '0' and c <= '9') x = x * 10 + (c ^ 48), c = RC(); return s == 1 ? (int)x : (int)(0-x); } inline void PC(char c){ if(OP == (int)sizeof OB) fwrite(OB, 1, OP, stdout), OP = 0; OB[OP++] = c; } inline void W(int n){ char s[24]; int p = 0; unsigned long long x = n; if(n < 0) PC('-'), x = 0 - x; do s[p++] = char('0' + x % 10), x /= 10; while(x); while(p) PC(s[--p]); } struct Flush { ~Flush(){ fwrite(OB, 1, OP, stdout); } } flush;</pre>
run	1 lines
<pre>g++ -std=c++20 -Dtsao -Wall -Wextra -Wshadow -fsanitize=address -std=gnu++20 -O2 -pipe \$1.cpp -o \$1 && ./\$1</pre>	
s.sh	9 lines
<pre>#!/usr/bin/env bash set -e for f in gen good bad; do g++ -std=c++20 -O2 \$f.cpp -o \$f; done while :; do ./gen > in ./good < in > out1 ./bad < in > out2 diff out1 out2 break done</pre>	

hash	1 lines
<pre>cpp -x c++ -std=c++20 -w -dD -P -fpreprocessed \$1.cpp tr -d '[:space:]' md5sum cut -c-6</pre>	
troubleshoot.txt	52 lines
Pre-submit: Write a few simple test cases if sample is not enough. Are time limits close? If so, generate max cases. Is the memory usage fine? Could anything overflow? Make sure to submit the right file.	
Wrong answer: Print your solution! Print debug output, as well. Are you clearing all data structures between test cases? Can your algorithm handle the whole range of input? Read the full problem statement again. Do you handle all corner cases correctly? Have you understood the problem correctly? Any uninitialized variables? Any overflows? Confusing N and M, i and j, etc.? Are you sure your algorithm works? What special cases have you not thought of? Are you sure the STL functions you use work as you think? Add some assertions, maybe resubmit. Create some testcases to run your algorithm on. Go through the algorithm for a simple case. Go through this list again. Explain your algorithm to a teammate. Ask the teammate to look at your code. Go for a small walk, e.g. to the toilet. Is your output format correct? (including whitespace) Rewrite your solution from the start or let a teammate do it.	
Runtime error: Have you tested all corner cases locally? Any uninitialized variables? Are you reading or writing outside the range of any vector? Any assertions that might fail? Any possible division by 0? (mod 0 for example) Any possible infinite recursion? Invalidated pointers or iterators? Are you using too much memory? Debug with resubmits (e.g. remapped signals, see Various).	
Time limit exceeded: Do you have any possible infinite loops? What is the complexity of your algorithm? Are you copying a lot of unnecessary data? (References) How big is the input and output? (consider scanf) Avoid vector, map. (use arrays/unordered_map) What do your teammates think about your algorithm?	
Memory limit exceeded: What is the max amount of memory your algorithm should need? Are you clearing all data structures between test cases?	
Mathematics (2) 2.1 Equations $ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ The extremum is given by $x = -b/2a$.	

$$\begin{aligned} ax + by &= e & x &= \frac{ed - bf}{ad - bc} \\ cx + dy &= f & y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

2.2 Recurrences

If $a_n = c_1 a_{n-1} + \dots + c_k a_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \dots - c_k$, there are $d_1, \dots, d_k$ s.t.

$$a_n = d_1 r_1^n + \dots + d_k r_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

2.3 Trigonometry

$$\begin{aligned} \sin(v + w) &= \sin v \cos w + \cos v \sin w \\ \cos(v + w) &= \cos v \cos w - \sin v \sin w \end{aligned}$$

$$\begin{aligned} \tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \end{aligned}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi) \end{aligned}$$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2} \sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b + c} \right)^2 \right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

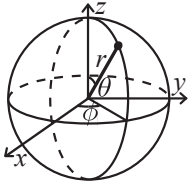
2.4.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° , $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

2.4.3 Spherical coordinates



$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \text{atan2}(y, x) \end{aligned}$$

2.5 Derivatives/Integrals

$$\begin{aligned} \frac{d}{dx} \arcsin x &= \frac{1}{\sqrt{1 - x^2}} & \frac{d}{dx} \arccos x &= -\frac{1}{\sqrt{1 - x^2}} \\ \frac{d}{dx} \tan x &= 1 + \tan^2 x & \frac{d}{dx} \arctan x &= \frac{1}{1 + x^2} \\ \int \tan ax &= -\frac{\ln |\cos ax|}{a} & \int x \sin ax &= \frac{\sin ax - ax \cos ax}{a^2} \\ \int e^{-x^2} &= \frac{\sqrt{\pi}}{2} \text{erf}(x) & \int x e^{ax} dx &= \frac{e^{ax}}{a^2} (ax - 1) \end{aligned}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \dots + n = \frac{n(n + 1)}{2}$$

$$1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(2n + 1)(n + 1)}{6}$$

$$1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n + 1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n + 1)(2n + 1)(3n^2 + 3n - 1)}{30}$$

2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty)$$

$$\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1)$$

$$\sqrt{1 + x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2 V(X) + b^2 V(Y).$$

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\text{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1 - p)^{n - k}$$

$$\mu = np, \sigma^2 = np(1 - p)$$

$\text{Bin}(n, p)$ is approximately $\text{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $Fs(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1 - p)^{k-1}, \, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \, \sigma^2 = \frac{1 - p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $Po(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, \, k = 0, 1, 2, \dots$$

$$\mu = \lambda, \, \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $U(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a + b}{2}, \, \sigma^2 = \frac{(b - a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $Exp(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi \mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j / π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1} \pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

Data structures (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th For multiset, change less to less_equal, but lower_bound will become upper_bound and you can only erase iterator element, and finding the index of an element. To get a map, change null.type. **Time:** $\mathcal{O}(\log N)$

```
using namespace __gnu_pbds;

template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

void example() {
    Tree<int> t, t2; t.insert(8);
    auto it = t.insert(10).first;
    assert(it == t.lower_bound(9));
    assert(t.order_of_key(10) == 1);
    assert(t.order_of_key(11) == 2);
    assert(*t.find_by_order(0) == 8);
    t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

```
140fd8, 6 lines
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
    const uint64_t C = (1ll)(4e18 * acos(0)) | 71;
    ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll, int, chash> h({}, {}, {}, {}, {
    1<<16});
```

Matrix.h

Description: Basic operations on square matrices. **Usage:** Matrix<int, 3> A; A.d = { {{ {1, 2, 3}}, { {4, 5, 6}}, { {7, 8, 9}}} }; array<int, 3> vec = {1, 2, 3}; vec = (A^N) * vec;

```
6ab5db, 26 lines
template<class T, int N> struct Matrix {
    typedef Matrix M;
    array<array<T, N>, N> d{};
    M operator*(const M& m) const {
        M a;
        rep(i, 0, N) rep(j, 0, N)
            rep(k, 0, N) a.d[i][j] += d[i][k]*m.d[k][j];
        return a;
    }
    array<T, N> operator*(const array<T, N>& vec) const {
        array<T, N> ret{};
        rep(i, 0, N) rep(j, 0, N) ret[i] += d[i][j] * vec[j];
        return ret;
    }
    M operator^(ll p) const {
        assert(p >= 0);
        M a, b(*this);
        rep(i, 0, N) a.d[i][i] = 1;
        while(p){
            if(p&1) a = a*b;
            b = b*b;
            p >>= 1;
        }
        return a;
    }
};
```

LineContainer.h

Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”). **Time:** $\mathcal{O}(\log N)$

```
8ec1c7, 30 lines
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a, b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b){ // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y){
        if(y == end()) return x->p = inf, 0;
        if(x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
}
```

```
void add(ll k, ll m){
    auto z = insert({k, m, 0}), y = z++, x = y;
    while(isect(y, z)) z = erase(z);
    if(x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while((y = x) != begin() && (--x)->p >= y->p)
        isect(x, erase(y));
}
ll query(ll x){
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
}
};
```

Lichao.h

Description: Li Chao tree on known x-coordinates; supports line/segment add.

Usage: LiChao tr(xs); tr.add({m, b}); tr.add({m, b}, l, r); tr.query(x);

Time: $\mathcal{O}(\log N)$ per line/query, $\mathcal{O}(\log^2 N)$ per segment

23b102, 42 lines

```
struct Line {
    ll m=0, b=0;
    ll operator()(ll x) const { return m*x+b; }
};
struct LiChao {
    const ll INF = 4e18;
    vector<ll> x; vector<Line> st;
    bool mn;
    LiChao(vector<ll> x, bool mn=true) : x(x), mn(mn){
        sort(all(this->x)); this->x.erase(unique(all(this->x)),
            this->x.end());
        assert(sz(this->x));
        st.assign(4*sz(this->x), {0, mn?INF:-INF});
    }
    bool better(Line a, Line b, int p){ return mn ? a(x[p])<b(x[p]) : a(x[p])>b(x[p]); }
    void add(Line f, int l, int r, int p){
        int m=(l+r)/2; bool a=better(f, st[p], l), b=better(f, st[p], m);
        if(b) swap(f, st[p]);
        if(r-l==1) return;
        if(a!=b) add(f, l, m, p*2); else add(f, m, r, p*2+1);
    }
    void add(Line f){ add(f, 0, sz(x), 1); }
    void add(Line f, int a, int b, int l, int r, int p){
        if(b<=l or r<=a) return;
        if(a<=l and r<=b) return add(f, l, r, p);
        int m=(l+r)/2;
        add(f, a, b, l, m, p*2); add(f, a, b, m, r, p*2+1);
    }
    void add(Line f, ll l, ll r){ // x in [l, r]
        int a=lower_bound(all(x), l)-x.begin(), b=upper_bound(all(x), r)-x.begin();
        if(a<b) add(f, a, b, 0, sz(x), 1);
    }
    ll query(int p, int l, int r, int q){
        ll ans=st[p](x[q]);
        if(r-l==1) return ans;
        int m=(l+r)/2; ll z=q<m?query(p*2, l, m, q):query(p*2+1, m, r, q);
        return mn?min(ans, z):max(ans, z);
    }
    ll query(ll q){
        q=lower_bound(all(x), q)-x.begin(); assert(q<sz(x));
        return query(1, 0, sz(x), q);
    }
};
```

Treap.h

Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.

Time: $\mathcal{O}(\log N)$

1754b4, 53 lines

```
struct Node {
    Node *l = 0, *r = 0;
    int val, y, c = 1;
    Node(int val) : val(val), y(rand()){
        void recalc();
    };

    int cnt(Node* n){ return n ? n->c : 0; }
    void Node::recalc(){ c = cnt(l) + cnt(r) + 1; }

    template<class F> void each(Node* n, F f){
        if(n){ each(n->l, f); f(n->val); each(n->r, f); }
    }

    pair<Node*, Node*> split(Node* n, int k){
        if(!n) return {};
        if(cnt(n->l) >= k){ // "n->val >= k" for lower_bound(k)
            auto [L, R] = split(n->l, k);
            n->l = R;
            n->recalc();
            return {L, n};
        }else{
            auto [L, R] = split(n->r, k - cnt(n->l) - 1); // and just "k"
            n->r = L;
            n->recalc();
            return {n, R};
        }
    }

    Node* merge(Node* l, Node* r){
        if(!l) return r;
        if(!r) return l;
        if(l->y > r->y){
            l->r = merge(l->r, r);
            return l->recalc(), l;
        }else{
            r->l = merge(l, r->l);
            return r->recalc(), r;
        }
    }

    Node* ins(Node* t, Node* n, int pos){
        auto [l, r] = split(t, pos);
        return merge(merge(l, n), r);
    }

    // Example application: move the range [l, r] to index k
    void move(Node& t, int l, int r, int k){
        Node *a, *b, *c;
        tie(a, b) = split(t, l); tie(b, c) = split(b, r - l);
        if(k <= l) t = merge(ins(a, b, k), c);
        else t = merge(a, ins(c, b, k - r));
    }
};
```

pre-seg.h

Description: Persistent segment tree: point assign and range sum on $[0, n)$.

Usage: Seg st(n); int root2=st.modify(root1, p, x); st.query(root2, l, r);

Time: $\mathcal{O}(\log n)$ per operation

b8993e, 27 lines

```
struct Seg {
    struct Node { int ch[2]{}; ll val=0; };
    vector<Node> t{};
    int n;
```

```
Seg(int n, int q = 0) : n(n){ if(q) t.reserve(1+q*(__lg(n)+1)); }

int modify(int u, int l, int r, int p, ll x){
    int v = sz(t); t.pb(t[u]);
    if(r-l == 1) return t[v].val=x, v;
    int m=(l+r)/2, d=p>=m;
    if(d) t[v].ch[1]=modify(t[v].ch[1], m, r, p, x);
    else t[v].ch[0]=modify(t[v].ch[0], l, m, p, x);
    t[v].val=t[t[v].ch[0]].val+t[t[v].ch[1]].val;
    return v;
}

int modify(int root, int p, ll x){
    assert(0<=p and p<n); return modify(root, 0, n, p, x);
}

ll query(int u, int l, int r, int a, int b) const {
    if(!u or b<=l or r<=a) return 0;
    if(a<=l and r<=b) return t[u].val;
    int m=(l+r)/2;
    return query(t[u].ch[0], l, m, a, b)+query(t[u].ch[1], m, r, a, b);
}

ll query(int root, int l, int r) const {
    return query(root, 0, n, l, r);
}
};
```

MoQueries.h

Description: Mo ordering for half-open intervals. add/del get (index, end).

Time: $\mathcal{O}(N\sqrt{Q})$ moves

6d40ed, 53 lines

```
template<class Add, class Del, class Calc>
auto mo(vector<pii> Q, Add add, Del del, Calc calc, int blk=350){
    using T=decltype(calc());
    int L=0, R=0; vi ord(sz(Q)); vector<T> ans(sz(Q));
    iota(all(ord), 0);
    sort(all(ord), [&](int i, int j){
        auto [a, b]=Q[i]; auto [c, d]=Q[j];
        return pii(a/blk, b^-(a/blk&1)) < pii(c/blk, d^-(c/blk&1));
    });
    for(int i:ord){
        auto [l, r]=Q[i];
        while(L>l) add(--L, 0);
        while(R<r) add(R++, 1);
        while(L<l) del(L++, 0);
        while(R>r) del(--R, 1);
        ans[i]=calc();
    }
    return ans;
}

template<class Add, class Del, class Calc>
auto moTree(vector<array<int, 2>> Q, vector<vi> &g, Add add, Del del, Calc calc, int root=0, int blk=350){
    using T=decltype(calc());
    int n=sz(g), ti=n, pos[2]={root, root};
    vi ord(sz(Q)), I(n+2), L(n), R(n), in(n), par(n), tmp(n);
    vector<T> ans(sz(Q)); add(root, 0), in[root]=1;
    auto dfs=[&](auto self, int u, int p, int d)->void {
        par[u]=p; L[u]=ti;
        if(d) I[u]=ti++;
        for(int v:g[u]) if(v!=p) self(self, v, u, !d);
        if(!d) I[u]=ti++;
        R[u]=ti;
    };
    dfs(dfs, root, -1, 0); iota(all(ord), 0);
    sort(all(ord), [&](int i, int j){
        auto a=Q[i], b=Q[j];
```

```
        return pii(I[a[0]]/blk, I[a[1]]^(~(I[a[0]]/blk&1)) <
            pii(I[b[0]]/blk, I[b[1]]^(~(I[b[0]]/blk&1)));
    });
    for(int qi:ord) rep(e, 0, 2){
        int &a=pos[e], b=Q[qi][e], k=0;
        auto step=[&](int c){
            if(in[c]) del(a, e), in[a]=0; else add(c, e), in[c]=1;
            a=c;
        };
        while(not(L[b]<=L[a] and R[a]<=R[b])) tmp[k++]=b, b=par[b];
        while(a!=b) step(par[a]);
        while(k) step(tmp[--k]);
        if(e) ans[qi]=calc();
    }
    return ans;
}
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

c9b7b0, 17 lines

```
struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for(int i = sz(a); i--;) (val *= x) += a[i];
        return val;
    }
    void diff(){
        rep(i, 1, sz(a)) a[i-1] = i*a[i];
        a.pop_back();
    }
    void divroot(double x0){
        double b = a.back(), c; a.back() = 0;
        for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
        a.pop_back();
    }
};
```

PolyRoots.h

7f59a3, 28 lines

Description: Finds the distinct real roots of a polynomial in [xmin, xmax].
Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

```
vector<double> polyRoots(Poly p, double xmin, double xmax){
    while(sz(p.a) > 1 && abs(p.a.back()) < 1e-14) p.a.pop_back();
    if(sz(p.a) <= 1) return {};
    if(sz(p.a) == 2){
        double x = -p.a[0]/p.a[1];
        return xmin <= x && x <= xmax ? vector<double>{x} : vector<double>{};
    }
    Poly der = p; der.diff();
    auto at = polyRoots(der, xmin, xmax);
    at.insert(at.begin(), xmin); at.pb(xmax);
    vector<double> ret;
    auto add = [&](double x){
        if(abs(p(x)) < 1e-8 && (ret.empty() || abs(ret.back()-x) > 1e-7)) ret.pb(x);
    };
    rep(i, 0, sz(at)){
        add(at[i]);
        if(i+1 == sz(at)) break;
        double l = at[i], r = at[i+1], fl = p(l), fr = p(r);
        if((fl > 0) == (fr > 0) || abs(fl) < 1e-8 || abs(fr) < 1e-8) continue;
        rep(it, 0, 80){
```

```
        double m = (l+r)/2, fm = p(m);
        if((fl > 0) == (fm > 0)) l = m, fl = fm;
        else r = m;
    }
    add((l+r)/2);
}
return ret;
}
```

PolyInterpolate.h

08bf48, 13 lines

Description: Given n points $(x[i], y[i])$, computes an $n-1$ -degree polynomial p that passes through them: $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$.
Time: $\mathcal{O}(n^2)$

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n){
    vd res(n), temp(n);
    rep(k, 0, n-1) rep(i, k+1, n)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    rep(k, 0, n) rep(i, 0, n){
        res[i] += y[k] * temp[i];
        swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
    return res;
}
```

BerlekampMassey.h

96548b, 20 lines

Description: Recovers any n -order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}
Time: $\mathcal{O}(N^2)$

```
"../number-theory/ModPow.h"
vector<ll> berlekampMassey(vector<ll> s){
    int n = sz(s), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    rep(i, 0, n){ ++m;
        ll d = s[i] % mod;
        rep(j, 1, L+1) d = (d + C[j] * s[i - j]) % mod;
        if(!d) continue;
        T = C; ll coef = d * modpow(b, mod-2) % mod;
        rep(j, m, n) C[j] = (C[j] - coef * B[j - m]) % mod;
        if(2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }

    C.resize(L + 1); C.erase(C.begin());
    for(ll& x : C) x = (mod - x) % mod;
    return C;
}
```

LinearRecurrence.h

f4e444, 26 lines

Description: Generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
Usage: linearRec({0, 1}, {1, 1}, k) // k 'th Fibonacci number
Time: $\mathcal{O}(n^2 \log k)$

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k){
    int n = sz(tr);
```

```
auto combine = [&](Poly a, Poly b){
    Poly res(n * 2 + 1);
    rep(i, 0, n+1) rep(j, 0, n+1)
        res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for(int i = 2 * n; i > n; --i) rep(j, 0, n)
        res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
    res.resize(n + 1);
    return res;
};

Poly pol(n + 1), e(pol);
pol[0] = e[1] = 1;

for(++k; k; k /= 2){
    if(k % 2) pol = combine(pol, e);
    e = combine(e, e);
}

ll res = 0;
rep(i, 0, n) res = (res + pol[i + 1] * S[i]) % mod;
return res;
}
```

Operation.h

5cfbc1, 295 lines

Description: Formal power series modulo mod. Supports multiplication, inverse, shift, derivative, integral, logarithm, exponential, power, square root, multipoint evaluation, interpolation, division, compositional inverse, and composition.
Requires: NTT, modular arithmetic helpers, fac, ivf, Jacobi, QuadraticResidue
Time: $\mathcal{O}(N \log N)$ for multiplication and $\mathcal{O}(N \log^2 N)$ for most advanced operations

```
// maybe need fac ivf
// == PART HASH ==
#define fi(s, n) for (int i = (int)(s); i < (int)(n); i++)
#define neg(x) (x ? mod - x : 0)
#define V (*this)
template <int MAXN, int RT> // MAXN = 2^k
struct Poly : vector<int> { // coefficients in [0, P)
    using vector<int>::vector;
    static inline NTT<MAXN, RT> ntt;
    int n() const { return (int)size(); } // n() >= 1
    Poly(const Poly &p, int m) : vector<int>(m) { copy_n(p.data(), min(p.n(), m), data()); }
    Poly &irev() { return reverse(data(), data() + n()), V; }
    Poly &isz(int m) { return resize(m), V; }
    // == PART HASH ==
    Poly &iadd(const Poly &rhs) { // SCOPE HASH
        fi(0, n()) add(V[i], rhs[i]);
        return V; // need n() == rhs.n()
    }
    Poly &imul(int k) { // SCOPE HASH
        fi(0, n()) V[i] = mul(V[i], k);
        return V;
    }
    Poly &mul_xk(int m){ // SCOPE HASH
        if(m<0) erase(begin(), begin()+(-m));
        else if(m>0) insert(begin(), m, 0);
        return V;
    }
    Poly Mul(const Poly &rhs) const { // SCOPE HASH
        int m = 1;
        while (m < n() + rhs.n() - 1) m <= 1;
        assert(m <= MAXN);
        Poly X(V, m), Y(rhs, m);
        ntt(X, m); ntt(Y, m);
        fi(0, m) X[i] = mul(X[i], Y[i]);
```

```

    ntt(X, m, true);
    return X.isz(n()) + rhs.n() - 1);
}
Poly Inv() const { // SCOPE HASH
    if (n() == 1) return {inv(V[0])};
    int m = 1; // need V[0] != 0, 2*sz<=MAXN
    while (m < n() * 2) m <= 1;
    assert(m <= MAXN);
    Poly Xi = Poly(V, (n() + 1) / 2).Inv().isz(m);
    Poly Y(V, m);
    ntt(Xi, m); ntt(Y, m);
    fi(0, m) {
        Xi[i] = mul(Xi[i], (2 - mul(Xi[i], Y[i])));
        add(Xi[i], mod);
    }
    ntt(Xi, m, true);
    return Xi.isz(n());
}
// == PART HASH ==
Poly &shift_inplace(const int &c) { // need fac[], ivf[]
    int n = V.n(); // 2 * sz <= MAXN
    for (int i = 0; i < n; i++) V[i] = mul(V[i], fac[i]);
    Poly g(n);
    int cp = 1;
    for (int i = 0; i < n; i++){
        g[i] = mul(cp, ivf[i]);
        cp = mul(cp, c);
    }
    V = V.irev().Mul(g).isz(n).irev();
    for (int i = 0; i < n; i++) V[i] = mul(V[i], ivf[i]);
    return V;
}
Poly Shift(const int &c) const { return Poly(V).shift_inplace(c); }
// == PART HASH ==
Poly Dx() const {
    Poly ret(n() - 1);
    fi(0, ret.n()) ret[i] = mul(i+1, V[i+1]);
    return ret.isz(max(1, ret.n()));
}
Poly Sx() const {
    Poly ret(n() + 1);
    fi(0, n()) ret[i + 1] = mul(inv(i+1), V[i]);
    return ret;
}
// == PART HASH ==
Poly Ln() const { // V[0] == 1, 2*sz<=MAXN
    return Dx().Mul(Inv()).Sx().isz(n());
}
Poly Exp() const { // V[0] == 0, 2*sz<=MAXN
    if (n() == 1) return {1};
    Poly X = Poly(V, (n() + 1) / 2).Exp().isz(n());
    Poly Y = X.Ln();
    Y[0] = mod - 1;
    fi(0, n()){
        Y[i] = V[i] - Y[i];
        add(Y[i], mod);
    }
    return X.Mul(Y).isz(n());
}
// == PART HASH ==
// M := P(P - 1). If k >= M, k := k % M + M.
Poly Pow(ll k) const { // 2*sz<=MAXN
    int nz = 0;
    while (nz < n() && !V[nz]) nz++;
    if (nz < min(k, (ll)n()) >= n()) return Poly(n());
    if (!k) return Poly(Poly{1}, n());
    Poly X(data() + nz, data() + nz + n() - nz * k);
    const int c = po(X[0], k % (mod - 1));

```

```

    return X.Ln().imul(k % mod).Exp().imul(c).irev().isz(n()).irev();
}
// == PART HASH ==
Poly _tmul(int nn, const Poly &rhs) const {
    Poly Y = Mul(rhs).isz(n() + nn - 1);
    return Poly(Y.data() + n() - 1, Y.data() + Y.n());
}
vector<int> _eval(const vector<int> &x, const vector<Poly> &up) const { // SCOPE HASH
    const int m = (int)x.size();
    if (!m) return {};
    vector<Poly> down(m * 2);
    down[1] = Poly(up[1]).irev().isz(n()).Inv().irev()._tmul(m, V);
    fi(2, m * 2) down[i] = up[i ^ 1]._tmul(up[i].n() - 1, down[i / 2]);
    vector<int> y(m);
    fi(0, m) y[i] = down[m + i][0];
    return y;
}
static vector<Poly> _treel(const vector<int> &x) { // SCOPE HASH
    const int m = (int)x.size();
    vector<Poly> up(m * 2);
    fi(0, m) up[m + i] = {neg(x[i]), 1};
    for (int i = m - 1; i > 0; --i) up[i] = up[i * 2].Mul(up[i * 2 + 1]);
    return up;
}
vector<int> Eval(const vector<int> &x) const {
    auto up = _treel(x); // 2^17, 1.8s
    return _eval(x, up);
}
// == PART HASH ==
static Poly Interpolate(const vector<int> &x, const vector<int> &y) { // SCOPE HASH
    const int m = (int)x.size(); // 2^17, 2.3s
    vector<Poly> up = _treel(x), down(m * 2);
    vector<int> z = up[1].Dx()._eval(x, up);
    fi(0, m) z[i] = mul(y[i], inv(z[i]));
    fi(0, m) down[m + i] = {z[i]};
    for (int i = m - 1; i > 0; --i)
        down[i] = down[i * 2].Mul(up[i * 2 + 1]).iadd(down[i * 2 + 1].Mul(up[i * 2]));
    return down[1];
}
pair<Poly, Poly> DivMod(const Poly &rhs) const { // SCOPE HASH
    if (n() < rhs.n()) return {{0}, V}; // 5e5, 0.9s
    const int m = n() - rhs.n() + 1;
    Poly X(rhs); // (rhs.)back() != 0
    X.irev().isz(m);
    Poly Y(V); Y.irev().isz(m);
    Poly Q = Y.Mul(X.Inv()).isz(m).irev();
    X = rhs.Mul(Q), Y = V;
    fi(0, n()) add(Y[i], -X[i]);
    return {Q, Y.isz(max(1, rhs.n() - 1))};
}
// == PART HASH ==
Poly _Sqrt() const { // Jacobi(V[0], P) = 1
    if (n() == 1) return {QuadraticResidue(V[0], mod)};
    Poly X = Poly(V, (n() + 1) / 2)._Sqrt().isz(n());
    return X.iadd(Mul(X.Inv()).isz(n())).imul(mod / 2 + 1);
}
Poly Sqrt() const { // SCOPE HASH
    Poly a; // 2 * sz <= MAXN
    bool has = 0;
    for (int i = 0; i < n(); i++) {

```

```

        if (V[i]) has = 1;
        if (has) a.push_back(V[i]);
    }
    if (!has) return V;
    if ((n() + a.n()) % 2 || Jacobi(a[0], mod) != 1) {
        return Poly();
    }
    a = a.isz((n() + a.n()) / 2)._Sqrt();
    int sz = a.n();
    a.isz(n());
    rotate(a.begin(), a.begin() + sz, a.end());
    return a;
}
// == PART HASH ==
Poly Shift_samples(int c, int m) {
    // V = \sum f(n) x^n, return f(c), ..., f(c+m-1); 2^19, 2s
    Poly A=V;
    Poly Q(n()+1), S(n());
    int nw=1;
    fi(0, n()+1) { Q[i] = mul(1-2*(i&1), nw); nw = mul(nw, n()-i, inv(i+1)); }
    nw=1;
    fi(0, n()) { S[i] = mul(1-2*(i&1), nw); nw = mul(nw, c-i, inv(i+1)); }
    S=S.Shift(1);
    fi(0,n()) if(i&1) {S[i] = mul(S[i],-1); add(S[i], mod);};
    Poly C=Mul(Q).mul_xk(-n());
    auto tmp=Q.isz(m).Inv();
    C=C.imul(-1).Mul(tmp).isz(m).mul_xk(n());
    A=A.isz(n()+m).iadd(C).irev().isz(n()+m);
    return A.Mul(S).mul_xk(-n()+1).isz(m+1).irev();
}
// == PART HASH ==
Poly power_projection(Poly wt, int m) { // SCOPE HASH
    assert(n() == wt.n()); // 4*sz <= MAXN
    if (!n()) return Poly(m + 1, 0);
    if (V[0] != 0) {
        int c = V[0];
        V[0] = 0;
        Poly A = V.power_projection(wt, m);
        fi(0, m + 1) A[i] = mul(A[i], fac[i]);
        Poly B(m + 1);
        int pow = 1;
        fi(0, m + 1) {B[i] = mul(pow, ivf[i]); pow = mul(pow, c);
            // inv. of fac
        }
        A = A.Mul(B).isz(m + 1);
        fi(0, m + 1) A[i] = mul(A[i], fac[i]);
        return A;
    }
    int n = 1;
    while (n < V.n()) n *= 2;
    isz(n), wt.isz(n).irev();
    int k = 1;
    Poly p(wt, 2 * n), q(V, 2 * n);
    q.imul(mod - 1);
    while (n > 1) {
        Poly r(2 * n * k);
        fi(0, 2 * n * k) r[i] = (i % 2 == 0 ? q[i] : neg(q[i]));
        Poly pq = p.Mul(r).isz(4 * n * k);
        Poly qq = q.Mul(r).isz(4 * n * k);
        fi(0, 2 * n * k) {
            add(pq[2 * n * k + i], p[i]);
            add(qq[2 * n * k + i], (q[i] + r[i]) % mod);
        }
        fill(p.begin(), p.end(), 0);
        fill(q.begin(), q.end(), 0);
        for(int j = 0; j < 2 * k; j++) fi(0, n / 2) {
            p[n * j + i] = pq[(2 * n) * j + (2 * i + 1)];
            q[n * j + i] = qq[(2 * n) * j + (2 * i + 0)];

```

```

    }
    n /= 2; k *= 2;
}
Poly ans(k);
fi(0, k) ans[i] = p[2 * i];
return ans.irev().isz(m + 1);
}
Poly FPSinv() { // SCOPE HASH
    const int n = V.n() - 1; // 2^17, 4s
    if (n == -1) return {};
    assert(V[0] == 0);
    if (n == 0) return V;
    assert(V[1] != 0);
    int c = V[1], ic = inv(c);
    imul(ic);
    Poly wt(n + 1);
    wt[n] = 1;
    Poly A = V.power_projection(wt, n);
    Poly g(n);
    fi(1, n + 1) g[n - i] = mul(n, A[i], inv(i));
    g = g.Pow(neg(inv(n)));
    g.insert(g.begin(), 0);
    int pow = 1;
    fi(0, g.n()) {g[i] = mul(g[i], pow); pow = mul(pow, ic);}
    return g;
}
// == PART HASH ==
Poly TMul(const Poly &rhs) const { // this[i] - rhs[j] = k;
    return Poly(V).irev().Mul(rhs).isz(n()).irev();
}
Poly comp_rec(int n, int k, Poly Q){ // SCOPE HASH
    if (n == 1) {
        Poly p(2 * k);
        irev();
        fi(0, k) p[2 * i] = V[i];
        return p;
    }
    Poly R(2 * n * k);
    fi(0, 2 * n * k) R[i] = (i % 2 == 0 ? Q[i] : neg(Q[i]));
    Poly QQ = Q.Mul(R).isz(4 * n * k);
    fi(0, 2 * n * k)
        add(QQ[2 * n * k + i], (Q[i] + R[i]) % mod);
    Poly nxt_Q(2 * n * k);
    for(int j = 0; j < 2 * k; j++) fi(0, n / 2)
        nxt_Q[n * j + i] = QQ[(2 * n) * j + (2 * i + 0)];
    Poly nxt_p = comp_rec(n / 2, k * 2, nxt_Q);
    Poly pq(4 * n * k);
    for(int j = 0; j < 2 * k; j++) fi(0, n / 2)
        add(pq[(2 * n) * j + (2 * i + 1)], nxt_p[n * j + i]);
    Poly p(2 * n * k);
    fi(0, 2 * n * k) add(p[i], pq[2 * n * k + i]);
    pq.pop_back();
    Poly x = pq.TMul(R);
    fi(0, 2 * n * k) add(p[i], x[i]);
    return p;
}
Poly FPScomp(Poly g) { // solves V(g(x))
    int sz = 1; // 2^17, 5s
    while(sz < n() || sz < g.n()) sz <= 1;
    return isz(sz), comp_rec(sz, 1, g.imul(mod-1).isz(2 * sz)).
        isz(sz).irev();
}
// == PART HASH ==
};
#undef fi
#undef V
#undef neg
using Poly_t = Poly<1 << 21, 3>;

```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is eps . Works equally well for maximization with a small change in the code. See Ternary-Search.h in the Various chapter for a discrete version.

Usage: double func(double x){ return 4*x+.3*x*x; }

double xmin = gss(-1000, 1000, func);

Time: $\mathcal{O}(\log((b - a)/\epsilon))$

31d45b, 14 lines

```

double gss(double a, double b, double (*f)(double)){
    double r = (sqrt(5)-1)/2, eps = 1e-7;
    double x1 = b - r*(b-a), x2 = a + r*(b-a);
    double f1 = f(x1), f2 = f(x2);
    while(b-a > eps)
        if(f1 < f2){ //change to > to find maximum
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r*(b-a); f1 = f(x1);
        }else{
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r*(b-a); f2 = f(x2);
        }
    return a;
}

```

HillClimbing.h

Description: Poor man's optimization for unimodal functions.

8eeaf, 14 lines

typedef array<double, 2> P;

```

template<class F> pair<double, P> hillClimb(P start, F f){
    pair<double, P> cur(f(start), start);
    for(double jmp = 1e9; jmp > 1e-20; jmp /= 2){
        rep(j, 0, 100) rep(dx, -1, 2) rep(dy, -1, 2){
            P p = cur.second;
            p[0] += dx*jmp;
            p[1] += dy*jmp;
            cur = min(cur, make_pair(f(p), p));
        }
    }
    return cur;
}

```

Simplex.h

Description: Maximizes $c^T x$ subject to $Ax \leq b$ and $x \geq 0$.

Duality: min $b^T y$ subject to $A^T y \geq c$, $y \geq 0$; finite optima agree. Returns {value, x}; value is -inf if infeasible and inf if unbounded.

Time: $\mathcal{O}(NM * \#pivots)$, exponential in the worst case

1589b0, 65 lines

```

typedef double T;
typedef vector<T>vd;
typedef vector<vd>vvd;
const T eps = 1e-8, inf = 1 / .0;
#define MP make_pair
#define ltj(X) if(s == -1 or MP(X[j], N[j]) < MP(X[s], N[s])) s = j

```

```

struct LPSolver{
    int m, n;
    vi N, B;
    vvd D;
    LPSolver(const vvd &A, const vd &b, const vd &c):
        m(sz(b)), n(sz(c)), N(n + 1), B(m), D(m + 2, vd(n + 2)){
        rep(i, 0, m) rep(j, 0, n) D[i][j] = A[i][j];
        rep(i, 0, m) B[i] = n + i, D[i][n] = -1, D[i][n + 1] = b[i];
        rep(j, 0, n) N[j] = j, D[m][j] = -c[j];
        N[n] = -1;
    }
}

```

```

D[m + 1][n] = 1;
}
void pivot(int r, int s){
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i, 0, m + 2) if(i != r and abs(D[i][s]) > eps){
        T *b = D[i].data(), inv2 = b[s] * inv;
        rep(j, 0, n + 2) b[j] -= a[j] * inv2;
        b[s] = a[s] * inv2;
    }
    rep(j, 0, n + 2) if(j != s) D[r][j] *= inv;
    rep(i, 0, m + 2) if(i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
}
bool simplex(int phase){
    int x = m + phase - 1;
    for(;;){
        int s = -1;
        rep(j, 0, n + 1) if(N[j] != -phase) ltj(D[x]);
        if(D[x][s] >= -eps) return 1;
        int r = -1;
        rep(i, 0, m){
            if(D[i][s] <= eps) continue;
            if(r == -1 or MP(D[i][n + 1] / D[i][s], B[i]) <
                MP(D[r][n + 1] / D[r][s], B[r])) r = i;
        }
        if(r == -1) return 0;
        pivot(r, s);
    }
}
pair<T, vd> solve(){
    int r = 0;
    rep(i, 1, m) if(D[i][n + 1] < D[r][n + 1]) r = i;
    if(D[r][n + 1] < -eps){
        pivot(r, n);
        if(!simplex(2) or D[m + 1][n + 1] < -eps) return {-inf, {}};
        rep(i, 0, m) if(B[i] == -1){
            int s = 0;
            rep(j, 1, n + 1) ltj(D[i]);
            pivot(i, s);
        }
    }
    bool ok = simplex(1);
    vd x(n);
    rep(i, 0, m) if(B[i] < n) x[B[i]] = D[i][n + 1];
    return {ok ? D[m][n + 1] : inf, x};
}
};

```

4.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.

Time: $\mathcal{O}(N^3)$

24b41d, 15 lines

```

double det(vector<vector<double>> a){
    int n = sz(a); double res = 1;
    rep(i, 0, n){
        int b = i;
        rep(j, i+1, n) if(fabs(a[j][i]) > fabs(a[b][i])) b = j;
        if(i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if(res == 0) return 0;
        rep(j, i+1, n){
            double v = a[j][i] / a[i][i];
            if(v != 0) rep(k, i+1, n) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}

```

```
}

IntDeterminant.h
Description: Calculates determinant using modular arithmetics. Modulos
can also be removed to get a pure-integer version.
Time:  $\mathcal{O}(N^3)$ 
```

```
2dc1e6, 19 lines

11 det(vector<vector<ll>> a, ll mod){
    int n = sz(a); ll ans = 1;
    for(auto &r : a) for(ll &x : r) x = (x%mod+mod)%mod;
    rep(i, 0, n){
        rep(j, i+1, n){
            while(a[j][i] != 0){ // gcd step
                ll t = a[i][i] / a[j][i];
                if(t){
                    rep(k, i, n) a[i][k] = (a[i][k]-a[j][k]*t)%mod;
                }
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if(!ans) return 0;
    }
    return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Solves A*x=b. Returns rank, x, or -1, if inconsistent.

Time: $\mathcal{O}(N^2M)$

```
9a2cdf, 37 lines

typedef vector<double>vd;
const double eps = 1e-12;

pair<int, vd> solveLinear(vector<vd>A, vd b, int m = -1){
    int n = sz(A), rank = 0, br = 0, bc = 0;
    if(n) m = sz(A[0]);
    else if(m == -1) m = 0;
    vi col(m);
    iota(all(col), 0);
    rep(i, 0, n){
        double v, bv = 0;
        rep(r, i, n) rep(c, i, m) if((v = abs(A[r][c])) > bv)
            br = r, bc = c, bv = v;
        if(bv <= eps){
            rep(j, i, n) if(abs(b[j]) > eps) return {-1, {}};
            break;
        }
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j, 0, n) swap(A[j][i], A[j][bc]);
        bv = 1 / A[i][i];
        rep(j, i + 1, n){
            double z = A[j][i] * bv;
            b[j] -= z * b[i];
            rep(k, i + 1, m) A[j][k] -= z * A[i][k];
        }
        rank++;
    }
    vd x(m);
    for(int i = rank; i--){
        b[i] /= A[i][i];
        x[col[i]] = b[i];
        rep(j, 0, i) b[j] -= A[j][i] * b[i];
    }
    return {rank, x};
}
```

SolveLinear2.h

Description: Solves A*x=b. Undetermined x[i] are NaN. Returns rank, x, or -1, if inconsistent.

Time: $\mathcal{O}(NMmin(N,M))$

```
46b7a2, 34 lines

pair<int, vd> solveLinear2(vector<vd>A, vd b, int m = -1){
    int n = sz(A), r = 0;
    if(n) m = sz(A[0]);
    else if(m == -1) m = 0;
    vi col;
    rep(c, 0, m){
        if(r == n) break;
        int p = r;
        rep(i, r, n) if(abs(A[i][c]) > abs(A[p][c])) p = i;
        if(abs(A[p][c]) <= eps) continue;
        swap(A[p], A[r]);
        swap(b[p], b[r]);
        double z = A[r][c];
        rep(j, c, m) A[r][j] /= z;
        b[r] /= z;
        rep(i, 0, n) if(i != r){
            z = A[i][c];
            rep(j, c, m) A[i][j] -= z * A[r][j];
            b[i] -= z * b[r];
        }
        col.pb(c);
        r++;
    }
    rep(i, r, n) if(abs(b[i]) > eps) return {-1, {}};
    vd x(m, numeric_limits<double>::quiet_NaN());
    vector<bool>pivot(m);
    for(int c : col) pivot[c] = 1;
    rep(i, 0, r){
        bool unique = 1;
        rep(c, 0, m) if(!pivot[c] and abs(A[i][c]) > eps) unique = 0;
        if(unique) x[col[i]] = b[i];
    }
    return {r, x};
}
```

SolveLinearBinary.h

Description: Solves A*x=b over F2. Returns rank, x, or -1, .

Time: $\mathcal{O}(N^2M)$

```
353e56, 29 lines

typedef bitset<1000>bs;

pair<int, bs> solveLinear(vector<bs>A, vi b, int m){
    int n = sz(A), rank = 0, br;
    assert(m <= sz(bs()));
    vi col(m);
    iota(all(col), 0);
    rep(i, 0, n){
        for(br = i; br < n; br++) if(A[br].any()) break;
        if(br == n){
            rep(j, i, n) if(b[j]) return {-1, {}};
            break;
        }
        int bc = A[br]._Find_next(i - 1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j, 0, n) if(A[j][i] != A[j][bc]) A[j].flip(i), A[j].flip(bc);
        rep(j, i + 1, n) if(A[j][i]) b[j] ^= b[i], A[j] ^= A[i];
        rank++;
    }
    bs x;
    for(int i = rank; i--){
```

```
        if(!b[i]) continue;
        x[col[i]] = 1;
        rep(j, 0, i) b[j] ^= A[j][i];
    }
    return {rank, x};
}
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.

Time: $\mathcal{O}(n^3)$

```
ebfff6, 35 lines

int matInv(vector<vector<double>>& A){
    int n = sz(A); vi col(n);
    vector<vector<double>> tmp(n, vector<double>(n));
    rep(i, 0, n) tmp[i][i] = 1, col[i] = i;

    rep(i, 0, n){
        int r = i, c = i;
        rep(j, i, n) rep(k, i, n)
            if(fabs(A[j][k]) > fabs(A[r][c]))
                r = j, c = k;
        if(fabs(A[r][c]) < 1e-12) return i;
        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
        rep(j, 0, n)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        double v = A[i][i];
        rep(j, i+1, n){
            double f = A[j][i] / v;
            A[j][i] = 0;
            rep(k, i+1, n) A[j][k] -= f*A[i][k];
            rep(k, 0, n) tmp[j][k] -= f*tmp[i][k];
        }
        rep(j, i+1, n) A[i][j] /= v;
        rep(j, 0, n) tmp[i][j] /= v;
        A[i][i] = 1;
    }

    for(int i = n-1; i > 0; --i) rep(j, 0, i){
        double v = A[j][i];
        rep(k, 0, n) tmp[j][k] -= v*tmp[i][k];
    }

    rep(i, 0, n) rep(j, 0, n) A[col[i]][col[j]] = tmp[i][j];
    return n;
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d,p,q,b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, 1 \leq i \leq n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.

If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.
Time: $\mathcal{O}(N)$

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b){
    int n = sz(b); vi tr(n);
    rep(i, 0, n-1){
        if(abs(diag[i]) < 1e-9 * abs(super[i])){ // diag[i] == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if(i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[+i] = 1;
        }else{
            diag[i+1] -= super[i]*sub[i]/diag[i];
            b[i+1] -= b[i]*sub[i]/diag[i];
        }
    }
    for(int i = n; i--;){
        if(tr[i]){
            swap(b[i], b[i-1]);
            diag[i-1] = diag[i];
            b[i] /= super[i-1];
        }else{
            b[i] /= diag[i];
            if(i) b[i-1] -= b[i]*super[i-1];
        }
    }
    return b;
}
```

4.4 Fourier transforms

FastFourierTransform.h
Description: `fft(a)` computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n , `reverse(start+1, end)`, FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use `NTT/FFTMod`.
Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a){
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for(static int k = 2; k < n; k *= 2){
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        rep(i, k, 2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i, 0, n) if(i < rev[i]) swap(a[i], a[rev[i]]);
    for(int k = 1; k < n; k *= 2)
        for(int i = 0; i < n; i += 2 * k) rep(j, 0, k){
            C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
}
vd conv(const vd& a, const vd& b){
    if(a.empty() || b.empty()) return {};
    vd res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
    vector<C> in(n), out(n);
    copy(all(a), begin(in));
```

```
    rep(i, 0, sz(b)) in[i].imag(b[i]);
    fft(in);
    for(C& x : in) x *= x;
    rep(i, 0, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    rep(i, 0, sz(res)) res[i] = imag(out[i]) / (4 * n);
    return res;
}
```

FastFourierTransformMod.h
Description: Exact convolution modulo M using three NTT primes and CRT. Supports transform lengths up to 2^{21} .
Time: $\mathcal{O}(N \log N)$

```
typedef vector<ll> vl;
namespace conv_mod {
template<int P> ll pw(ll a, ll n){
    ll r = 1;
    for(; n; n >>= 1, a = a*a%P) if(n&1) r = r*a%P;
    return r;
}
template<int P, int G> void ntt(vl& a, bool inv){
    int n = sz(a);
    for(int i = 1, j = 0; i < n; i++){
        int bit = n >> 1;
        for(; j&bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if(i < j) swap(a[i], a[j]);
    }
    for(int len = 2; len <= n; len <= 1){
        ll w1 = pw<P>(G, (P-1)/len);
        if(inv) w1 = pw<P>(w1, P-2);
        for(int i = 0; i < n; i += len){
            ll w = 1;
            rep(j, 0, len/2){
                ll u = a[i+j], v = a[i+j+len/2]*w%P;
                a[i+j] = (u+v)%P;
                a[i+j+len/2] = (u-v+P)%P;
                w = w*w1%P;
            }
        }
    }
    if(inv){
        ll ni = pw<P>(n, P-2);
        for(ll &x : a) x = x*ni%P;
    }
}
template<int P, int G> vl conv(const vl& a, const vl& b, int n,
    int s){
    vl x(n), y(n);
    rep(i, 0, sz(a)) x[i] = a[i]%P;
    rep(i, 0, sz(b)) y[i] = b[i]%P;
    ntt<P, G>(x, 0); ntt<P, G>(y, 0);
    rep(i, 0, n) x[i] = x[i]*y[i]%P;
    ntt<P, G>(x, 1); x.resize(s);
    return x;
}
}
template<int M> vl convMod(const vl& a, const vl& b){
    if(a.empty() || b.empty()) return {};
    const ll p0 = 998244353, p1 = 1004535809, p2 = 469762049;
    int s = sz(a)+sz(b)-1, n = 1;
    while(n < s) n <= 1;
    auto x = conv_mod::conv<p0, 3>(a, b, n, s);
    auto y = conv_mod::conv<p1, 3>(a, b, n, s);
    auto z = conv_mod::conv<p2, 3>(a, b, n, s);
    ll i01 = conv_mod::pw<p1>(p0, p1-2);
    ll p01 = p0*p1, i012 = conv_mod::pw<p2>(p01%p2, p2-2);
```

```
    vl ret(s);
    rep(i, 0, s){
        ll b1 = (y[i]-x[i]+p1)%p1*i01%p1;
        ll b2 = (z[i]-(x[i]+p0*b1)%p2+p2)%p2*i012%p2;
        ret[i] = (x[i]+(__int128)p0*b1+(__int128)p01*b2)%M;
    }
    return ret;
}
```

NumberTheoreticTransform.h
Description: NTT/convolution mod 998244353; inputs in $[0, mod)$, size up to 2^{23} .
Time: $\mathcal{O}(N \log N)$

```
template<int MAXN, ll P, ll RT>
struct NTT {
    ll w[MAXN];
    ll mpow(ll a, ll n) {
        ll re = 1;
        while (n) {
            if (n & 1) re = re * a % P;
            a = a * a % P;
            n >>= 1;
        }
        return re;
    }
    ll minv(ll a) {
        return mpow(a, P - 2);
    }
    NTT() {
        ll dw = mpow(RT, (P - 1) / MAXN);
        w[0] = 1;
        FOR (i, 1, MAXN - 1) w[i] = w[i - 1] * dw % P;
    }
    void bitrev(ll *a, int n) {
        int i = 0;
        for (int j = 1; j < n - 1; ++j) {
            for (int k = n >> 1; (i ^= k) < k; k >>= 1);
            if (j < i) swap(a[i], a[j]);
        }
    }
    void operator() (ll *a, int n, bool inv = false) {
        bitrev(a, n);
        for (int L = 2; L <= n; L <= 1) {
            int dx = MAXN / L, dl = L >> 1;
            for (int i = 0; i < n; i += L) {
                for (int j = i, x = 0; j < i + dl; ++j, x += dx)
                    {
                        ll tmp = a[j + dl] * w[x] % P;
                        if ((a[j + dl] = a[j] - tmp) < 0) a[j + dl] += P;
                        if ((a[j] += tmp) >= P) a[j] -= P;
                    }
            }
        }
        if (inv) {
            reverse(a + 1, a + n);
            ll invn = minv(n);
            FOR (i, 0, n - 1) a[i] = a[i] * invn % P;
        }
    }
};
```

FastSubsetTransform.h
Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.
Time: $\mathcal{O}(N \log N)$

```
void FST(vi& a, bool inv, int mod = 0){
    for(int n = sz(a), step = 1; step < n; step *= 2){
        for(int i = 0; i < n; i += 2 * step) rep(j, i, i+step){
            int &u = a[j], &v = a[j + step], x = u, y = v;
            if(mod){
                if(inv){ u = y - x; if(u < 0) u += mod; v = x; }
                else { u = y; v = x + y; if(v >= mod) v -= mod; }
            }
            else tie(u, v) = inv ? pii(y - x, x) : pii(y, x + y); //
                AND
                inv ? pii(v, u - v) : pii(u + v, u); // OR
                pii(u + v, u - v); // XOR
        }
    }
    if(inv) for(int& x : a) x /= sz(a); // XOR only
}
vi conv(vi a, vi b, int mod = 0){
    FST(a, 0, mod); FST(b, 0, mod);
    rep(i, 0, sz(a)) a[i] = mod ? (int)((ll)a[i]*b[i] % mod) : a[
        i]*b[i];
    FST(a, 1, mod); return a;
}
```

Number theory (5)

5.1 Modular arithmetic

5.1.1 Nice Primes

For NTT,
 $m = 9223372036737335297 = 549755813881 \times 2^{24} + 1, g = 3$ (very close to long long limit $2^{63} - 1$) For hashing we can also use $MOD = 1000000000000000003, 1000000000000000033, BASE = 911382323$

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.

"euclid.h"	ealac7, 19 lines
------------	------------------

```
const ll mod = 17; // change to something else
struct Mod {
    ll x;
    Mod(ll y) : Mod(y % mod + mod, 0){}
    Mod(ll y, int) : x(y < mod ? y : y - mod){}
    Mod operator+(Mod b){ return {x + b.x, 0}; }
    Mod operator-(Mod b){ return {x - b.x + mod, 0}; }
    Mod operator*(Mod b){ return {x * b.x % mod, 0}; }
    Mod operator/(Mod b){ return *this * invert(b); }
    Mod invert(Mod a){
        ll x, y, g = euclid(a.x, mod, x, y);
        assert(g == 1); return x;
    }
    Mod operator^(ll e){
        if(!e) return 1;
        Mod r = *this ^ (e / 2); r = r * r;
        return e&1 ? *this * r : r;
    }
};
```

ModInverse.h

Description: Inverses of [1, n) modulo a prime, assuming n <= mod.
Time: $\mathcal{O}(n)$

vector<ll> inverses(int n, ll mod){	c4afb5, 6 lines
-------------------------------------	-----------------

```
vector<ll> inv(n);
if(n > 1) inv[1] = 1;
```

```
rep(i, 2, n) inv[i] = mod - mod / i * inv[mod % i] % mod;
return inv;
}
```

ModPow.h

	1e790e, 9 lines
--	-----------------

```
const ll mod = 1000000007; // faster if const

ll modpow(ll b, ll e, ll m = mod){
    ll ans = 1;
    b %= m;
    for(; e; b = b * b % m, e /= 2)
        if(e & 1) ans = ans * b % m;
    return ans;
}
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a, 1, m) can be used to calculate the order of a .

Time: $\mathcal{O}(\sqrt{m})$	c040b8, 11 lines
--------------------------------------	------------------

```
ll modLog(ll a, ll b, ll m){
    ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while(j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if(e == b % m) return j;
    if(__gcd(m, e) == __gcd(m, b))
        rep(i, 2, n+2) if(A.count(e = e * f % m))
            return n * i - A[e];
    return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.
modsum(to, c, k, m) = $\sum_{i=0}^{to-1} (ki + c) \% m$. divsum is similar but for floored division.

Time: $\log(m)$, with a large constant.	5c5bc5, 16 lines
---	------------------

```
typedef unsigned long long ull;
ull sumsq(ull to){ return to / 2 * ((to-1) | 1); }
```

```
ull divsum(ull to, ull c, ull k, ull m){
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if(!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
```

```
ll modsum(ull to, ll c, ll k, ll m){
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Portable $a*b \pmod M$ for $0 \leq a, b < M < 2^{64}$.
Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log e)$ for modpow

	5cbaa6, 10 lines
--	------------------

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M){
    return (ull)((unsigned __int128)a * b % M);
}
ull modpow(ull a, ull e, ull M){
    ull r = 1;
    for(; e; e >>= 1, a = modmul(a, a, M))
        if(e & 1) r = modmul(r, a, M);
    return r;
}
```

```
}

ModSqrt.h
Description: Tonelli-Shanks algorithm for modular square roots. Finds  $x$  s.t.  $x^2 = a \pmod p$  ( $-x$  gives the other solution).
Time:  $\mathcal{O}(\log^2 p)$  worst case,  $\mathcal{O}(\log p)$  for most  $p$ 

"ModPow.h"
19a793, 24 lines

ll sqrt(ll a, ll p){
    a %= p; if(a < 0) a += p;
    if(a == 0) return 0;
    assert(modpow(a, (p-1)/2, p) == 1); // else no solution
    if(p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    ll s = p - 1, n = 2;
    int r = 0, m;
    while(s % 2 == 0)
        ++r, s /= 2;
    while(modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    ll x = modpow(a, (s + 1) / 2, p);
    ll b = modpow(a, s, p), g = modpow(n, s, p);
    for(;; r = m){
        ll t = b;
        for(m = 0; m < r && t != 1; ++m)
            t = t * t % p;
        if(m == 0) return x;
        ll gs = modpow(g, 1LL << (r - m - 1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

5.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 $\approx 1.5s$

	f9d471, 20 lines
--	------------------

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes(){
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vi pr = {2}, sieve(S+1); pr.reserve((size_t)(LIM/log(LIM)
        *1.1));
    vector<pii> cp;
    for(int i = 3; i <= S; i += 2) if(!sieve[i]){
        cp.push_back({i, i * i / 2});
        for(int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
    }
    for(int L = 1; L <= R; L += S){
        array<bool, S> block{};
        for(auto &[p, idx] : cp)
            for(int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
        rep(i, 0, min(S, R - L))
            if(!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for(int i : pr) isPrime[i] = 1;
    return pr;
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \pmod c$.

"ModMulLL.h"	60dcd1, 12 lines
--------------	------------------

```
bool isPrime(ull n){
    if(n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
```

```
s = __builtin_ctzll(n-1), d = n >> s;
for(ull a : A){ // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while(p != 1 && p != n - 1 && a % n && i--){
        p = modmul(p, p, n);
        if(p != n-1 && i != s) return 0;
    }
    return 1;
}
```

Factor.h
Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

```
"ModMulLL.h", "MillerRabin.h"                                d8d98d, 18 lines

ull pollard(ull n){
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x){ return modmul(x, x, n) + i; };
    while(t++ % 40 || __gcd(prd, n) == 1){
        if(x == y) x = ++i, y = f(x);
        if((q = modmul(prd, max(x, y) - min(x, y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}

vector<ull> factor(ull n){
    if(n == 1) return {};
    if(isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

5.3 Divisibility

euclid.h
Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `__gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

```
ll euclid(ll a, ll b, ll &x, ll &y){
    if(!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

CRT.h
Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

```
"euclid.h"                                                    04d93a, 7 lines

ll crt(ll a, ll m, ll b, ll n){
    if(n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

5.3.1 Bézout’s identity

For $a \neq 0, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$ax + by = d$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h
Description: Euler’s ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p$ prime $\Rightarrow \phi(p^k) = (p - 1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1}p_2^{k_2}...p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1}...(p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$.
 $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, \gcd(k, n) = 1} k = n\phi(n)/2, n > 1$
Euler’s thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat’s little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \ \forall a$.

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi(){
    rep(i, 0, LIM) phi[i] = i&1 ? i : i/2;
    for(int i = 3; i < LIM; i += 2) if(phi[i] == i)
        for(int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

ContinuedFractions.h
Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.
For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k) alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a ’s eventually become cyclic.
Time: $\mathcal{O}(\log N)$

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximated(d x, ll N){
    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
    for(;;){
        ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
            a = (ll)floor(y), b = min(a, lim),
            NP = b*P + LP, NQ = b*Q + LQ;
        if(a > b){
            // If b > a/2, we have a semi-convergent that gives us a
            // better approximation; if b = a/2, we *may* have one.
            // Return {P, Q} here for a more canonical approximation.
            return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
                make_pair(NP, NQ) : make_pair(P, Q);
        }
        if(abs(y = 1/(y - (d)a)) > 3*N){
            return {NP, NQ};
        }
        LP = P; P = NP;
        LQ = Q; Q = NQ;
    }
}
```

FracBinarySearch.h
Description: Given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: `fracBS([&f](Frac f){ return f.p>=3*f.q; }, 10);` // {1, 3}
Time: $\mathcal{O}(\log(N))$

```
struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N){
    bool dir = 1, A = 1, B = 1;
```

```
Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
if(f(lo)) return lo;
assert(f(hi));
while(A || B){
    ll adv = 0, step = 1; // move hi if dir, else lo
    for(int si = 0; step; (step *= 2) >= si){
        adv += step;
        Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
        if(abs(mid.p) > N || mid.q > N || dir == !f(mid)){
            adv -= step; si = 2;
        }
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);
    A = B; B = !adv;
}
return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$

with $m > n > 0, k > 0, m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$

$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

```
IntPerm.h
Description: Permutation -> integer conversion. (Not order preserving.)
Integer -> permutation can use a lookup table.
Time: O(n)
044568, 6 lines

int permToInt(vi& v){
    int use = 0, i = 0, r = 0;
    for(int x:v) r = r * ++i + __builtin_popcount(use & ~(1<<x)),
                use |= 1 << x; // (note: minus, not ~!)
    return r;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^\infty g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g\in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k|n} f(k)\phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \; p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n-k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

<i>n</i>	0	1	2	3	4	5	6	7	8	9	20	50	100
<i>p</i> (<i>n</i>)	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

6.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_kp^k + \dots + n_1p + n_0$ and $m = m_kp^k + \dots + m_1p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$.

6.2.3 Binomials

```
multinomial.h
Description: Computes \binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \dots k_n!}.
a0a312, 5 lines

ll multinomial(vi& v){
    ll c = 1, m = v.empty() ? 1 : v[0];
    rep(i, 1, sz(v)) rep(j, 0, v[i]) c = c * ++m / (j+1);
    return c;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able).

$$B[0,\dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$$

Sums of powers:

$$\sum_{i=1}^n n^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\begin{aligned} \sum_{i=m}^\infty f(i) &= \int_m^\infty f(x)dx - \sum_{k=1}^\infty \frac{B_k}{k!} f^{(k-1)}(m) \\ &\approx \int_m^\infty f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m)) \end{aligned}$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$\begin{aligned} c(n,k) &= c(n-1,k-1) + (n-1)c(n-1,k), \; c(0,0) = 1 \\ \sum_{k=0}^n c(n,k)x^k &= x(x+1)\dots(x+n-1) \end{aligned}$$

$$\begin{aligned} c(8,k) &= 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1 \\ c(n,2) &= 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots \end{aligned}$$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j :s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j :s s.t. $\pi(j) \geq j$, k j :s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$ For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1n_2\dots n_kn^{k-2}$
with degrees d_i : $(n-2)!/((d_1-1)!\dots(d_n-1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1, \; C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \; C_{n+1} = \sum C_i C_{n-i}$$

$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

BellmanFord.h

Description: Single-source shortest paths with negative cycles. Unreachable vertices have INF; vertices after a negative cycle have -INF.

Time: $\mathcal{O}(VE)$

278b44, 24 lines

```
struct BellmanFord{
    struct Edge{ int u, v; ll w; };
    const ll INF = 1LL << 62;
    int n;
    vector<Edge>ed;
    BellmanFord(int N) : n(N){}
    void addEdge(int u, int v, ll w){
        ed.pb({u, v, w});
    }
    pair<vector<ll>, vi> calc(int s){
        vector<ll>d(n, INF);
        vi par(n, -1);
        d[s] = 0;
        rep(i, 0, n){
            for(auto [u, v, w] : ed) if(d[u] != INF){
                ll nd = max(-INF, d[u] + w);
                if(nd < d[v]) d[v] = i == n - 1 ? -INF : nd, par[v] = u;
            }
        }
        rep(i, 0, n) for(auto [u, v, w] : ed)
            if(d[u] == -INF) d[v] = -INF;
        return {d, par};
    }
};
```

FloydWarshall.h

Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m , where $m[i][j]$ = inf if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j , inf if no path, or -inf if the path goes through a negative-weight cycle.

Time: $\mathcal{O}(N^3)$

531245, 12 lines

```
const ll inf = 1LL << 62;
void floydWarshall(vector<vector<ll>>& m){
    int n = sz(m);
    rep(i, 0, n) m[i][i] = min(m[i][i], 0LL);
    rep(k, 0, n) rep(i, 0, n) rep(j, 0, n)
        if(m[i][k] != inf && m[k][j] != inf){
            auto newDist = max(m[i][k] + m[k][j], -inf);
            m[i][j] = min(m[i][j], newDist);
        }
    rep(k, 0, n) if(m[k][k] < 0) rep(i, 0, n) rep(j, 0, n)
        if(m[i][k] != inf && m[k][j] != inf) m[i][j] = -inf;
}
```

7.2 Network flow

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}(V^2\sqrt{E})$

0ae1d4, 48 lines

```
struct PushRelabel {
    struct Edge {
        int dest, back;
        ll f, c;
    };
    vector<vector<Edge>> g;
    vector<ll> ec;
    vector<Edge*> cur;
    vector<vi> hs; vi H;
    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n){}
```

```
void addEdge(int s, int t, ll cap, ll rcap=0){
    if(s == t) return;
    g[s].push_back({t, sz(g[t]), 0, cap});
    g[t].push_back({s, sz(g[s])-1, 0, rcap});
}

void addFlow(Edge& e, ll f){
    Edge &back = g[e.dest][e.back];
    if(!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
    e.f += f; e.c -= f; ec[e.dest] += f;
    back.f -= f; back.c += f; ec[back.dest] -= f;
}

ll calc(int s, int t){
    int v = sz(g); H[s] = v; ec[t] = 1;
    vi co(2*v); co[0] = v-1;
    rep(i, 0, v) cur[i] = g[i].data();
    for(Edge& e : g[s]) addFlow(e, e.c);

    for(int hi = 0;;){
        while(hs[hi].empty()) if(!hi--) return -ec[s];
        int u = hs[hi].back(); hs[hi].pop_back();
        while(ec[u] > 0) // discharge u
            if(cur[u] == g[u].data() + sz(g[u])){
                H[u] = 1e9;
                for(Edge& e : g[u]) if(e.c && H[u] > H[e.dest]+1)
                    H[u] = H[e.dest]+1, cur[u] = &e;
                if(++co[H[u]], !--co[hi] && hi < v)
                    rep(i, 0, v) if(hi < H[i] && H[i] < v)
                        --co[H[i]], H[i] = v + 1;
                hi = H[u];
            } else if(cur[u]->c && H[u] == H[cur[u]->dest]+1)
                addFlow(*cur[u], min(ec[u], cur[u]->c));
            else ++cur[u];
        }
    }
    bool leftOfMinCut(int a){ return H[a] >= sz(g); }
};
```

MinCostMaxFlow.h

Description: Successive shortest path min-cost max-flow using potentials and a PBDS heap. Call setpi before maxflow if initial residual edges can have negative costs. Negative-cost cycles are not supported.

Usage: Prefer when the number of augmentations is small, or each shortest path can push a large bottleneck flow. Usually the simplest and fastest default for ordinary MCMF. Initial negative edges need setpi; negative cycles need the lower-bound transformation described below, or CostScalingMCMF. PBDS requires GNU GCC/libstdc++.

Time: $\mathcal{O}(F\epsilon\log V)$, and $\mathcal{O}(VE)$ for setpi

fa77d4, 94 lines

```
const ll MCMF_INF = numeric_limits<ll>::max() / 4;
struct MCMF{
    struct edge{
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>>ed;
    vi seen;
    vector<ll>dist, pi;
    vector<edge*>par;
    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N){}
    void addEdge(int from, int to, ll cap, ll cost){
        if(from == to) return;
        ed[from].push_back(edge{from, to, sz(ed[to]), cap, cost, 0})
    );
```

```
        ed[to].push_back(edge{to, from, sz(ed[from]) - 1, 0, -cost, 0});
    }
    void path(int s){
        fill(all(seen), 0);
        fill(all(dist), MCMF_INF);
        fill(all(par), nullptr);
        dist[s] = 0;
        using PQ = __gnu_pbds::priority_queue<pair<ll, int>, greater<pair<ll, int>>>;
        PQ pq;
        vector<PQ::point_iterator> its(N);
        pq.push({0, s});
        while(!pq.empty()){
            int v = pq.top().S;
            pq.pop();
            seen[v] = 1;
            ll di = dist[v] + pi[v];
            for(edge &e : ed[v]){
                if(seen[e.to]) continue;
                if(e.cap - e.flow <= 0) continue;
                ll val = di - pi[e.to] + e.cost;
                if(val < dist[e.to]){
                    dist[e.to] = val;
                    par[e.to] = &e;
                    if(its[e.to] == pq.end()){
                        its[e.to] = pq.push({dist[e.to], e.to});
                    } else{
                        pq.modify(its[e.to], {dist[e.to], e.to});
                    }
                }
            }
        }
        rep(i, 0, N){
            if(dist[i] != MCMF_INF){
                pi[i] = min(pi[i] + dist[i], MCMF_INF);
            }
        }
    }
    pair<ll, ll> maxflow(int s, int t){
        ll totflow = 0, totcost = 0;
        while(path(s), seen[t]){
            ll fl = MCMF_INF;
            for(edge *x = par[t]; x; x = par[x->from]){
                fl = min(fl, x->cap - x->flow);
            }
            totflow += fl;
            for(edge *x = par[t]; x; x = par[x->from]){
                x->flow += fl;
                ed[x->to][x->rev].flow -= fl;
            }
        }
        rep(i, 0, N){
            for(edge &e : ed[i]){
                totcost += e.cost * e.flow;
            }
        }
        return {totflow, totcost / 2};
    }
}

// Negative edges: run setpi(s); rerun after changing source or flow.
// Negative cycles: use CostScalingMCMF.
void setpi(int s){
    fill(all(pi), MCMF_INF);
    pi[s] = 0;
    int it = N, ch = 1;
    ll v;
    while(ch-- && it--){
        rep(i, 0, N){
```

```

    if(pi[i] == MCMF_INF) continue;
    for(edge &e : ed[i]){
        if(e.cap - e.flow <= 0) continue;
        if((v = pi[i] + e.cost) < pi[e.to]){
            pi[e.to] = v;
            ch = 1;
        }
    }
}
}
assert(it >= 0); // negative cost cycle
}
};

```

MinCostMaxFlowCostScaling.h

Description: Min-cost max-flow using Dinic followed by epsilon cost-scaling push-relabel. Supports negative costs and negative-cost cycles. Costs are multiplied by N internally; make sure this and the final answer fit in ll.

Usage: Prefer for large total flow, many unit-capacity edges, integer costs, negative edges/cycles, or min-cost circulation. SSP is usually faster for only a few augmentations. Not suitable for floating-point costs. For pure circulation, start with any feasible flow and run only the cost-scaling phase; do not invent s, t or run maxFlow.

Time: $\mathcal{O}(V^2E + VE\log(VC))$, where C is the maximum absolute cost

61108e, 147 lines

```

const ll CS_INF = numeric_limits<ll>::max() / 4;
struct CostScalingMCMF{
    struct edge{
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>>>ed;
    vector<ll>excess, pi;
    vi ptr, inq;
    CostScalingMCMF(int N) : N(N), ed(N), excess(N), pi(N), ptr(N), inq(N){}
    // Supports negative edges/cycles with finite capacities.
    // No negative-cost self-loops or unbounded instances.
    void addEdge(int from, int to, ll cap, ll cost){
        if(from == to){
            assert(cost >= 0);
            return;
        }
        assert(-CS_INF / N <= cost && cost <= CS_INF / N);
        cost *= N;
        ed[from].push_back(edge{from, to, sz(ed[to]), cap, cost, 0});
        ed[to].push_back(edge{to, from, sz(ed[from]) - 1, 0, -cost, 0});
    }
    ll residual(edge &e){
        return e.cap - e.flow;
    }
    void push(edge &e, ll fl){
        e.flow += fl;
        ed[e.to][e.rev].flow -= fl;
        excess[e.from] -= fl;
        excess[e.to] += fl;
    }
    bool admissible(edge &e){
        return residual(e) > 0 && pi[e.from] + e.cost - pi[e.to] < 0;
    }
    void relabel(int v, ll eps){
        ll best = -CS_INF;

```

```

        ptr[v] = -1;
        rep(i, 0, sz(ed[v])){
            edge &e = ed[v][i];
            if(residual(e) > 0 && best < pi[e.to] - e.cost){
                best = pi[e.to] - e.cost;
                ptr[v] = i;
            }
        }
        assert(ptr[v] != -1);
        pi[v] = best - eps;
    }
    void refine(ll eps){
        fill(all(excess), 0);
        fill(all(ptr), 0);
        fill(all(inq), 0);
        rep(i, 0, N){
            for(edge &e : ed[i]){
                if(admissible(e)) push(e, residual(e));
            }
        }
        queue<int> q;
        rep(i, 0, N){
            if(excess[i] > 0){
                q.push(i);
                inq[i] = 1;
            }
        }
        while(!q.empty()){
            int v = q.front();
            q.pop();
            inq[v] = 0;
            while(excess[v] > 0){
                if(ptr[v] == sz(ed[v])) relabel(v, eps);
                edge &e = ed[v][ptr[v]];
                if(!admissible(e)){
                    ++ptr[v];
                    continue;
                }
                ll fl = min(excess[v], residual(e));
                int to = e.to;
                push(e, fl);
                if(excess[to] > 0 && !inq[to]){
                    q.push(to);
                    inq[to] = 1;
                }
            }
        }
    }
    ll maxFlow(int s, int t){
        vi level(N), work(N);
        auto bfs = [&]() {
            fill(all(level), -1);
            queue<int> q;
            level[s] = 0;
            q.push(s);
            while(!q.empty()){
                int v = q.front();
                q.pop();
                for(edge &e : ed[v]){
                    if(residual(e) > 0 && level[e.to] == -1){
                        level[e.to] = level[v] + 1;
                        q.push(e.to);
                    }
                }
            }
        };
        return level[t] != -1;
    };
    auto dfs = [&](auto &&self, int v, ll pushed) -> ll {
        if(v == t or pushed == 0) return pushed;

```

```

        for(int &i = work[v]; i < sz(ed[v]); ++i){
            edge &e = ed[v][i];
            if(level[e.to] != level[v] + 1) continue;
            ll fl = self(self, e.to, min(pushed, residual(e)));
            if(fl == 0) continue;
            e.flow += fl;
            ed[e.to][e.rev].flow -= fl;
            return fl;
        }
        return 0;
    };
    ll flow = 0;
    while(bfs()){
        fill(all(work), 0);
        while(ll fl = dfs(dfs, s, CS_INF)) flow += fl;
    }
    return flow;
}
pair<ll, ll> maxflow(int s, int t){
    // Find any max flow, then reduce cost by circulation.
    ll totflow = maxFlow(s, t);
    ll eps = 0;
    rep(i, 0, N){
        for(edge &e : ed[i]){
            eps = max(eps, abs(e.cost));
        }
    }
    while(eps > 0){
        refine(eps);
        if(eps == 1) break;
        eps = max(1LL, eps / 4);
    }
    __int128 totcost = 0;
    rep(i, 0, N){
        for(edge &e : ed[i]){
            totcost += (__int128)e.cost * e.flow;
        }
    }
    return {totflow, (ll)(totcost / 2 / N)};
}
};

```

Dinic.h

Description: Flow algorithm with complexity $\mathcal{O}(VE\log U)$ where $U = \max|cap|$. $\mathcal{O}(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $\mathcal{O}(\sqrt{VE})$ for bipartite matching.

d79769, 42 lines

```

struct Dinic{
    struct Edge{
        int to, rev;
        ll c, oc;
        ll flow(){ return max(oc - c, 0LL); } // if you need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>>>adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n){}
    void addEdge(int a, int b, ll c, ll rcap = 0){
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f){
        if(v == t || !f) return f;
        for(int &i = ptr[v]; i < sz(adj[v]); i++){
            Edge &e = adj[v][i];
            if(lvl[e.to] == lvl[v] + 1)
                if(ll p = dfs(e.to, t, min(f, e.c))){
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
    }
};

```

```
    }
    return 0;
}
ll calc(int s, int t){
    ll flow = 0; q[0] = s;
    rep(L, 0, 31) do { // 'int L=30' maybe faster for random data
        lvl = ptr = vi(sz(q));
        int qi = 0, qe = lvl[s] = 1;
        while(qi < qe and !lvl[t]){
            int v = q[qi++];
            for(Edge e : adj[v])
                if(!lvl[e.to] and e.c >> (30 - L))
                    q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
        }
        while(ll p = dfs(s, t, LLONG_MAX)) flow += p;
    }while(lvl[t]);
    return flow;
}
bool leftOfMinCut(int a){ return lvl[a] != 0; }
};
```

MinCut.h
Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

```
GlobalMinCut.h
Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
Time:  $\mathcal{O}(V^3)$ 
8b0e19, 21 lines
```

```
pair<int, vi> globalMinCut(vector<vi> mat){
    pair<int, vi> best = {INT_MAX, {}};
    int n = sz(mat);
    vector<vi> co(n);
    rep(i, 0, n) co[i] = {i};
    rep(ph, 1, n){
        vi w = mat[0];
        size_t s = 0, t = 0;
        rep(it, 0, n-ph){ //  $\mathcal{O}(V^2) \rightarrow \mathcal{O}(E \log V)$  with prio. queue
            w[t] = INT_MIN;
            s = t, t = max_element(all(w)) - w.begin();
            rep(i, 0, n) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
        co[s].insert(co[s].end(), all(co[t]));
        rep(i, 0, n) mat[s][i] += mat[t][i];
        rep(i, 0, n) mat[i][s] = mat[s][i];
        mat[0][t] = INT_MIN;
    }
    return best;
}
```

GomoryHu.h
Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Time: $\mathcal{O}(V)$ Flow Computations

```
"PushRelabel.h"
0418b3, 13 lines
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed){
    vector<Edge> tree;
    vi par(N);
    rep(i, 1, N){
        PushRelabel D(N); // Dinic also works
        for(Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
```

```
        tree.push_back({i, par[i], D.calc(i, par[i])});
        rep(j, i+1, N)
            if(par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
    }
    return tree;
}
```

7.3 Matching
hopcroftKarp.h
Description: Maximum bipartite matching. $r[v]$ is the left match of right v .
Time: $\mathcal{O}(Esqrt(V))$

```
f50330, 40 lines
struct HopcroftKarp{
    int n, m;
    vector<vi>g;
    vi l, r, d, q;
    HopcroftKarp(int N, int M): n(N), m(M), g(N), q(N){}
    void addEdge(int u, int v){
        g[u].pb(v);
    }
    bool dfs(int u){
        int z = exchange(d[u], 0) + 1;
        for(int v : g[u]){
            if(r[v] == -1 or (d[r[v]] == z and dfs(r[v]))) {
                l[u] = v;
                r[v] = u;
                return 1;
            }
        }
        return 0;
    }
    pair<int, vi> calc(){
        int ans = 0;
        l.assign(n, -1);
        r.assign(m, -1);
        for(;;){
            int s = 0, t = 0;
            bool found = 0;
            d.assign(n, 0);
            rep(i, 0, n) if(l[i] == -1) q[t++] = i, d[i] = 1;
            while(s < t){
                int u = q[s++];
                for(int v : g[u]){
                    if(r[v] == -1) found = 1;
                    else if(!d[r[v]]) d[r[v]] = d[u] + 1, q[t++] = r[v];
                }
            }
            if(!found) return {ans, r};
            rep(i, 0, n) if(l[i] == -1) ans += dfs(i);
        }
    }
};
```

DFSMatching.h
Description: Simple bipartite matching. $r[v]$ is the left match of right v .
Time: $\mathcal{O}(VE)$

```
2847a4, 29 lines
struct DFSMatching{
    int n, m;
    vector<vi>g;
    vi r, vis;
    DFSMatching(int N, int M): n(N), m(M), g(N){}
    void addEdge(int u, int v){
        g[u].pb(v);
    }
    bool dfs(int u){
        if(vis[u]) return 0;
        vis[u] = 1;
```

```
        for(int v : g[u]){
            if(r[v] == -1 or dfs(r[v])){
                r[v] = u;
                return 1;
            }
        }
        return 0;
    }
    pair<int, vi> calc(){
        int ans = 0;
        r.assign(m, -1);
        rep(i, 0, n){
            vis.assign(n, 0);
            ans += dfs(i);
        }
        return {ans, r};
    }
};
```

MinimumVertexCover.h
Description: Minimum vertex cover in a bipartite graph. Returns left vertices as u and right vertices as $n+v$.
Time: $\mathcal{O}(Esqrt(V))$

```
"hopcroftKarp.h"
99da56, 23 lines
struct MinimumVertexCover : HopcroftKarp{
    using HopcroftKarp::HopcroftKarp;
    vi cover(){
        auto [cnt, mt] = calc();
        vector<bool>left(n, 1), seen(m);
        for(int u : mt) if(u != -1) left[u] = 0;
        vi q, ans;
        rep(i, 0, n) if(left[i]) q.pb(i);
        while(sz(q)){
            int u = q.back();
            q.pop_back();
            left[u] = 1;
            for(int v : g[u]) if(!seen[v] and mt[v] != -1){
                seen[v] = 1;
                q.pb(mt[v]);
            }
        }
        rep(i, 0, n) if(!left[i]) ans.pb(i);
        rep(i, 0, m) if(seen[i]) ans.pb(n + i);
        assert(sz(ans) == cnt);
        return ans;
    }
};
```

WeightedMatching.h
Description: Minimum-cost matching of every row to a distinct column. $N \leq M$.
Time: $\mathcal{O}(N^2M)$

```
fec40, 21 lines
template<class T> pair<T, vi> hungarian(const vector<vector<T>>
    &C){
    int i = sz(C), m = i ? sz(C[0]) : 0, c = 0, s, r;
    vector<T> dist(m), pot(m); vi match(i), rev(m, -1), cols(m),
        pre(m);
    T d = 0, nd, cost = 0;
    while(i--){
        rep(j, 0, m) dist[j] = C[i][j], cols[j] = j, pre[j] = i;
        for(s = 0;;){
            rep(j, s, m){
                c = cols[j], nd = dist[c] - pot[c];
                if(j == s or d > nd) d = nd, swap(cols[s], cols[j]);
            }
            if((r = rev[c = cols[s++]]) == -1) break;
            rep(j, 0, m) if(dist[j] > (nd = C[r][j] - C[r][c] + dist[
                c]))
```

```

    dist[j] = nd, pre[j] = r;
}
cost += dist[c];
while(s--) pot[cols[s]] = dist[cols[s]] - d;
for( r != i; swap(c, match[r])) r = rev[c] = pre[c];
}
return {cost, match};
}

```

GeneralMatching.h

Description: Maximum matching in a general undirected graph.

Time: $\mathcal{O}(N^3)$ ef29b7, 72 lines

```

struct GeneralMatching {
    int n, st;
    vector<vi> g;
    vi mt, even, groupBuf;
    vector<pii> nx;
    GeneralMatching(int n) : n(n), g(n), mt(n), even(n), groupBuf(n), nx(n){}
    void addEdge(int a, int b){
        g[a].pb(b); g[b].pb(a);
    }
    int group(int x){
        if(groupBuf[x] == -1 || even[groupBuf[x]] != st) return groupBuf[x];
        return groupBuf[x] = group(groupBuf[x]);
    }
    void rematch(int p, int b){
        int d = mt[p]; mt[p] = b;
        if(d == -1 || mt[d] != p) return;
        if(nx[p].S == -1){
            mt[d] = nx[p].F;
            rematch(nx[p].F, d);
        }else{
            rematch(nx[p].F, nx[p].S);
            rematch(nx[p].S, nx[p].F);
        }
    }
    bool augment(){
        even[st] = st; groupBuf[st] = -1; nx[st] = {-1, -1};
        queue<int> q; q.push(st);
        while(!q.empty()){
            int a = q.front(); q.pop();
            for(int b : g[a]){
                if(b == st) continue;
                if(mt[b] == -1){
                    mt[b] = a; rematch(a, b);
                    return 1;
                }
            }
            if(even[b] == st){
                int x = group(a), y = group(b), z = -1;
                if(x == y) continue;
                while(x != -1 || y != -1){
                    if(y != -1) swap(x, y);
                    if(nx[x] == pii(a, b)){
                        z = x;
                        break;
                    }
                }
                nx[x] = {a, b};
                x = group(nx[mt[x]].F);
            }
            for(int v : {group(a), group(b)}){
                while(v != z){
                    q.push(v); even[v] = st; groupBuf[v] = z;
                    v = group(nx[mt[v]].F);
                }
            }
        }
        return 1;
    }
};

```

```

    even[mt[b]] = st; nx[b] = {-1, -1};
    nx[mt[b]] = {a, -1}; groupBuf[mt[b]] = b;
    q.push(mt[b]);
}
}
return 0;
}
pair<int, vi> calc(){
    fill(all(mt), -1); fill(all(even), -1); fill(all(groupBuf), 0);
    int ans = 0;
    rep(i, 0, n){
        st = i;
        if(mt[i] == -1) ans += augment();
    }
    return {ans, mt};
}
};

```

7.4 DFS algorithms

SCC.h

Description: Iterative Kosaraju SCC. sccid is in reverse topological order.

Time: $\mathcal{O}(V + E)$ ff4399, 44 lines

```

struct SCC{
    int n, cnt = 0;
    vector<vi> g, rg;
    vi sccid;
    SCC(int n) : n(n), g(n), rg(n), sccid(n, -1){}
    void add_edge(int u, int v){
        g[u].pb(v);
        rg[v].pb(u);
    }
    int build(){
        vi it(n), order, st;
        vector<bool> vis(n);
        rep(s, 0, n) if(!vis[s]){
            st.pb(s);
            vis[s] = 1;
            while(sz(st)){
                int u = st.back();
                if(it[u] == sz(g[u])){
                    order.pb(u);
                    st.pop_back();
                }else{
                    int v = g[u][it[u]++];
                    if(!vis[v]) vis[v] = 1, st.pb(v);
                }
            }
        }
        cnt = 0;
        sccid.assign(n, -1);
        reverse(all(order));
        for(int s : order) if(sccid[s] == -1){
            st.clear();
            st.pb(s);
            sccid[s] = cnt;
            while(sz(st)){
                int u = st.back();
                st.pop_back();
                for(int v : rg[u]) if(sccid[v] == -1) sccid[v] = cnt, st.pb(v);
            }
            cnt++;
        }
        for(int &x : sccid) x = cnt - 1 - x;
        return cnt;
    }
};

```

```
};
```

EBCC.h

Description: Bridges and edge-biconnected components of an undirected graph.

Time: $\mathcal{O}(V + E)$ 9859a4, 45 lines

```

struct EBCC{
    int n, m = 0, nbcc = 0, ti = 0;
    vector<vector<pii>>g;
    vi in, low, id, is_bridge;
    EBCC(int n) : n(n), g(n){}
    void add_edge(int u, int v){
        g[u].pb({v, m});
        g[v].pb({u, m++});
    }
    int build(){
        nbcc = ti = 0; in.assign(n, -1); low.resize(n); id.assign(n, -1);
        is_bridge.assign(m, 0);
        vi it(n), par(n, -1), pe(n, -1), stk;
        rep(s, 0, n) if(in[s] == -1){
            in[s] = low[s] = ti++; stk.pb(s);
            while(sz(stk)){
                int u = stk.back();
                if(it[u] < sz(g[u])){
                    auto [v, e] = g[u][it[u]++];
                    if(e == pe[u]) continue;
                    if(in[v] == -1){
                        par[v] = u; pe[v] = e; in[v] = low[v] = ti++; stk.pb(v);
                    }else low[u] = min(low[u], in[v]);
                }else{
                    stk.pop_back();
                    if(pe[u] != -1){
                        int p = par[u];
                        low[p] = min(low[p], low[u]);
                        if(low[u] > in[p]) is_bridge[pe[u]] = 1;
                    }
                }
            }
        }
        rep(s, 0, n) if(id[s] == -1){
            id[s] = nbcc++; stk.pb(s);
            while(sz(stk)){
                int u = stk.back(); stk.pop_back();
                for(auto [v, e] : g[u]) if(!is_bridge[e] and id[v] == -1){
                    id[v] = id[s]; stk.pb(v);
                }
            }
        }
        return nbcc;
    }
};

```

Dominatortree.h

Description: Immediate dominators from source s. Unreachable nodes get -1.

Time: $\mathcal{O}((V + E)\alpha(V))$ 57ee93, 52 lines

```

struct DominatorTree {
    int n, id;
    vector<vector<int>> g, rg, bucket;
    vector<int> sdom, dom, vis, rev, pa, rt, mn, res;
    // dom[s] = s, dom[v] = -1 if s -> v not exists
    int query(int v, int x){
        if(rt[v] == v) return x ? -1 : v;
        int p = query(rt[v], 1);
    }
};

```

```
if(p == -1) return x ? rt[v] : mn[v];
if(sdom[mn[v]] > sdom[mn[rt[v]]])
    mn[v] = mn[rt[v]];
rt[v] = p;
return x ? p : mn[v];
}
void dfs(int v){
    vis[v] = id, rev[id] = v;
    rt[id] = mn[id] = sdom[id] = id, id++;
    for(int u : g[v]){
        if(vis[u] == -1) dfs(u), pa[vis[u]] = vis[v];
        rg[vis[u]].pb(vis[v]);
    }
}
void build(int s){
    id = 0;
    for(auto &v : rg) v.clear();
    for(auto &v : bucket) v.clear();
    fill(all(vis), -1); fill(all(dom), -1);
    dfs(s);
    for(int i = id - 1; ~i; --i){
        for(int u : rg[i])
            sdom[i] = min(sdom[i], sdom[query(u, 0)]);
        if(i) bucket[sdom[i]].pb(i);
        for(int u : bucket[i]){
            int p = query(u, 0);
            dom[u] = sdom[p] == i ? i : p;
        }
        if(i) rt[i] = pa[i];
    }
    fill(all(res), -1);
    for(int i = 1; i < id; ++i){
        if(dom[i] != sdom[i]) dom[i] = dom[dom[i]];
    }
    for(int i = 1; i < id; ++i)
        res[rev[i]] = rev[dom[i]];
    res[s] = s;
    for(int i = 0; i < n; ++i) dom[i] = res[i];
}
void add_edge(int u, int v){ g[u].pb(v); }
DominatorTree (int _n) : n(_n), id(0), g(n), rg(n),
    bucket(n), sdom(n), dom(n, -1), vis(n, -1),
    rev(n), pa(n), rt(n), mn(n), res(n){}
};
```

BiconnectedComponents.h

Description: Vertex-biconnected components and the block-cut forest. Original vertices are [0, n), component nodes are [n, n+nbcc).
Time: $\mathcal{O}(V + E)$

f8d87e, 56 lines

```
struct BCC{
    int n, m = 0, nbcc = 0, ti = 0;
    vector<vector<pii>>>g;
    vector<vi>tree, comp;
    vi in, low;
    BCC(int n): n(n), g(n){}
    void add_edge(int u, int v){
        g[u].pb({v, m});
        g[v].pb({u, m++});
    }
    void add_comp(int u, int v, vi &stk){
        int id = n + nbcc++;
        tree.pb({}); comp.pb({});
        while(1){
            int x = stk.back(); stk.pop_back();
            tree[id].pb(x), tree[x].pb(id), comp.back().pb(x);
            if(x == v) break;
        }
        tree[id].pb(u), tree[u].pb(id), comp.back().pb(u);
    }
};
```

BiconnectedComponents 2sat EdgeColoring

```
}
void build(){
    nbcc = ti = 0; in.assign(n, -1); low.resize(n);
    tree.assign(n, {}); comp.clear(); tree.reserve(n + m);
    vi it(n), par(n, -1), pe(n, -1), stk, dfs;
    rep(s, 0, n) if(in[s] == -1){
        if(g[s].empty()){
            int id = n + nbcc++;
            tree.pb({s}); tree[s].pb(id); comp.pb({s});
            in[s] = low[s] = ti++;
            continue;
        }
        in[s] = low[s] = ti++; stk.pb(s); dfs.pb(s);
        while(sz(dfs)){
            int u = dfs.back();
            if(it[u] < sz(g[u])){
                auto [v, e] = g[u][it[u]++];
                if(e == pe[u]) continue;
                if(in[v] == -1){
                    par[v] = u; pe[v] = e; in[v] = low[v] = ti++;
                    stk.pb(v); dfs.pb(v);
                }else low[u] = min(low[u], in[v]);
            }else{
                dfs.pop_back();
                if(pe[u] != -1){
                    int p = par[u];
                    low[p] = min(low[p], low[u]);
                    if(low[u] >= in[p]) add_comp(p, u, stk);
                }
            }
            stk.clear();
        }
    }
    bool is_cut(int u) const { return sz(tree[u]) > 1; }
    const vi &bcc(int id) const { return comp[id]; }
};
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c, ... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).
Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0, ~1, 2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars
Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

b33304, 65 lines

```
struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2*n){}

    int addVar(){ // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }

    void either(int f, int j){
        f = max(2*f, -1-2*f);
        j = max(2*j, -1-2*j);
        gr[f].push_back(j^1);
    }
};
```

```
gr[j].push_back(f^1);
}
void setValue(int x){ either(x, x); }

void atMostOne(const vi& li){ // (optional)
    if(sz(li) <= 1) return;
    int cur = ~li[0];
    rep(i, 2, sz(li)){
        int next = addVar();
        either(cur, ~li[i]);
        either(cur, next);
        either(~li[i], next);
        cur = ~next;
    }
    either(cur, ~li[1]);
}

bool solve(){
    vector<vi> rg(2*N); vi vis(2*N), ord, comp(2*N, -1);
    rep(i, 0, 2*N) for(int j : gr[i]) rg[j].pb(i);
    rep(s, 0, 2*N) if(!vis[s]){
        vector<pii> st = {{s, 0}}; vis[s] = 1;
        while(!st.empty()){
            int u = st.back().F;
            if(st.back().S < sz(rg[u])){
                int v = rg[u][st.back().S++];
                if(!vis[v]) vis[v] = 1, st.pb({v, 0});
            }else ord.pb(u), st.pop_back();
        }
    }
    int c = 0;
    for(int oi = sz(ord); oi--;){
        int s = ord[oi]; if(comp[s] != -1) continue;
        vi st = {s}; comp[s] = c;
        while(!st.empty()){
            int u = st.back(); st.pop_back();
            for(int v : rg[u]) if(comp[v] == -1) comp[v] = c, st.pb(v);
        }
        ++c;
    }
    values.resize(N);
    rep(i, 0, N){
        if(comp[2*i] == comp[2*i+1]) return 0;
        values[i] = comp[2*i] < comp[2*i+1];
    }
    return 1;
}
};
```

7.5 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Time: $\mathcal{O}(NM)$

e210e2, 31 lines

```
vi edgeColoring(int N, vector<pii> eds){
    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for(pii e : eds) ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vi> adj(N, vi(ncols, -1));
    for(pii e : eds){
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while(d = free[v], !loc[d] && (v = adj[u][d]) != -1)
```

```
loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
cc[loc[d]] = c;
for(int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
    swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
while(adj[fan[i]][d] != -1){
    int left = fan[i], right = fan[++i], e = cc[i];
    adj[u][e] = left;
    adj[left][e] = u;
    adj[right][e] = -1;
    free[right] = e;
}
adj[u][d] = fan[i];
adj[fan[i]][d] = u;
for(int y : {fan[0], u, end})
    for(int& z = free[y] = 0; adj[y][z] != -1; z++);
}
rep(i, 0, sz(eds))
    for(tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
return ret;
}
```

7.6 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}){
    if(!P.any()) { if(!X.any()) f(R); return; }
    auto q = (P | X)._Find_first();
    auto cands = P ^ ~eds[q];
    rep(i, 0, sz(eds)) if(cands[i]){
        R[i] = 1;
        cliques(eds, f, P ^ eds[i], X ^ eds[i], R);
        R[i] = P[i] = 0; X[i] = 1;
    }
}
```

MaximumClique.h

Description: Fast maximum clique for a symmetric bitset graph, $n \leq 200$.

```
typedef vector<bitset<200>> vb;
struct Maxclique {
    double limit=.025, pk=0;
    struct Vertex { int i, d=0; };
    using vv=vector<Vertex>;
    vb e; vv V; vector<vi> C; vi qmax, q, steps, old;
    void init(vv &r){
        if(r.empty()) return;
        for(auto &v:r) v.d=0;
        for(auto &v:r) for(auto u:r) v.d+=e[v.i][u.i];
        sort(all(r), [](auto a, auto b){return a.d>b.d;});
        int mx=r[0].d;
        rep(i, 0, sz(r)) r[i].d=min(i, mx)+1;
    }
    void expand(vv &R, int lev=1){
        steps[lev]+=steps[lev-1]-old[lev]; old[lev]=steps[lev-1];
        while(sz(R)){
            if(sz(q)+R.back().d<sz(qmax)) return;
            q.pb(R.back().i); vv T;
            for(auto v:R) if(e[R.back().i][v.i]) T.pb({v.i});
            if(sz(T)){
                if((double)steps[lev]++/++pk<limit) init(T);
                int j=0, mx=1, mn=max<int>(sz(qmax)-sz(q)+1, 1);
                C[1].clear(), C[2].clear();
```

```
for(auto v:T){
    int k=1;
    while(any_of(all(C[k]), [&](int i){return e[v.i][i];})
        ) ++k;
    if(k>mx) mx=k, C[mx+1].clear();
    if(k<mn) T[j++].i=v.i;
    C[k].pb(v.i);
}
if(j) T[j-1].d=0;
rep(k, mn, mx+1) for(int i:C[k]) T[j].i=i, T[j++].d=k;
expand(T, lev+1);
} else if(sz(q)>sz(qmax)) qmax=q;
q.pop_back(), R.pop_back();
}
}
vi maxClique(){ if(V.empty()) return {}; init(V), expand(V);
return qmax; }
Maxclique(vb e):e(e), C(sz(e)+2), steps(C.size()), old(steps)
{
    rep(i, 0, sz(e)) V.pb({i});
}
};
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertex-Cover.

7.7 Trees

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.

Time: $\mathcal{O}(|S| \log |S|)$

```
"LCA.h"
bb56cc, 22 lines

typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset){
    if(subset.empty()) return {};
    static vi rev; rev.resize(sz(lca.time));
    vi li = subset, &T = lca.time;
    auto cmp = [&](int a, int b){ return T[a] < T[b]; };
    sort(all(li), cmp);
    int m = sz(li)-1;
    rep(i, 0, m){
        int a = li[i], b = li[i+1];
        li.push_back(lca.lca(a, b));
    }
    sort(all(li), cmp);
    li.erase(unique(all(li)), li.end());
    rep(i, 0, sz(li)) rev[li[i]] = i;
    vpi ret = {pii(0, li[0])};
    rep(i, 0, sz(li)-1){
        int a = li[i], b = li[i+1];
        ret.emplace_back(rev[lca.lca(a, b)], b);
    }
    return ret;
}
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log(n)$ light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS.EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.

```
Time:  $\mathcal{O}\left((\log N)^2\right)$ 
".../data-structures/LazySegmentTree.h"
73ed04, 46 lines

template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, rt, pos;
    Node *tree;
    HLD(vector<vi> adj_){
        N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
        rt(N), pos(N), tree(new Node(0, N)){ dfsSz(0); dfsHld(0);
    }
    void dfsSz(int v){
        for(int& u : adj[v]){
            adj[u].erase(find(all(adj[u]), v));
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if(siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v){
        pos[v] = tim++;
        for(int u : adj[v]){
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B op){
        for(;; v = par[rt[v]]){
            if(pos[u] > pos[v]) swap(u, v);
            if(rt[u] == rt[v]) break;
            op(pos[rt[v]], pos[v] + 1);
        }
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void modifyPath(int u, int v, ll val){
        process(u, v, [&](int l, int r){ tree->add(l, r, val); });
    }
    ll queryPath(int u, int v){ // Modify depending on problem
        ll res = -inf;
        process(u, v, [&](int l, int r){
            res = max(res, tree->query(l, r));
        });
        return res;
    }
    ll querySubtree(int v){ // modifySubtree is similar
        return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
    }
};
```

LinkCutTree.h

Description: 1-based link-cut tree with path and rooted-subtree aggregates. Val and SVal need + and -; Val also needs reverse-order aggregation.

Usage: access(u): expose root-u path, splay u, and clear its right child.

chroot(u): make u the represented-tree root.

split(u, v): expose path u-v in v; its aggregate is o[v].sum.

vir stores virtual-child aggregates; access moves preferred children

into/out of vir. sub includes both aux children, vir, and sv.

Time: $\mathcal{O}(\log N)$ amortized

```
// 1-based
// = PART HASH =
template <typename Val, typename SVal> struct LCT {
    struct node {
        int pa, ch[2]; bool rev; int size;
        Val v, sum, rsum; SVal sv, sub, vir;
        node() : pa{0}, ch{0, 0}, rev{false}, size{1}, v{},
```

```
sum{}, rsum{}, sv{}, sub{}, vir{} {}
};
#define cur o[u]
#define lc cur.ch[0]
#define rc cur.ch[1]
vector<node> o;
bool is_root(int u) const {
    return o[cur.pa].ch[0]!=u && o[cur.pa].ch[1]!=u; }
bool is_rch(int u) const {
    return o[cur.pa].ch[1] == u && !is_root(u); }
void down(int u){
    for(int c : {lc, rc}) if(c){
        if(cur.rev) set_rev(c);
    }
    cur.rev = false;
}
void up(int u){
    cur.sum = o[lc].sum + cur.v + o[rc].sum;
    cur.rsum = o[rc].rsum + cur.v + o[lc].rsum;
    cur.sub = cur.vir + o[lc].sub + o[rc].sub + cur.sv;
    cur.size = o[lc].size + o[rc].size + 1;
}
void set_rev(int u){
    swap(lc, rc), swap(cur.sum, cur.rsum);
    cur.rev ^= 1;
}
// == PART HASH ==
void rotate(int u){
    int f = cur.pa, g = o[f].pa, l = is_rch(u);
    if(cur.ch[l ^ 1]) o[cur.ch[l ^ 1]].pa = f;
    if(not is_root(f)) o[g].ch[is_rch(f)] = u;
    o[f].ch[l] = cur.ch[l ^ 1], cur.ch[l ^ 1] = f;
    cur.pa = g, o[f].pa = u; up(f);
}
vector<int> stk;
void splay(int u){
    stk.clear(); stk.pb(u);
    while(not is_root(stk.back()))
        stk.push_back(o[stk.back()].pa);
    while(not stk.empty())
        down(stk.back()), stk.pop_back();
    for(int f = cur.pa; not is_root(u); f = cur.pa){
        if(!is_root(f))
            rotate(is_rch(u) == is_rch(f) ? f : u);
        rotate(u);
    }
    up(u);
}
void access(int x){
    for(int u = x, last = 0; u; u = cur.pa){
        splay(u);
        cur.vir = cur.vir + o[rc].sub - o[last].sub;
        rc = last; up(last = u);
    }
    splay(x);
}
int find_root(int u){
    int la = 0;
    for(access(u); u; u = lc) down(la = u);
    splay(la);
    return la;
}
void split(int x, int y){ chroot(x); access(y); }
void chroot(int u){ access(u); set_rev(u); }
// == PART HASH ==
LCT(int n = 0) : o(n + 1){ o[0].size = 0; }
void set_val(int u, const Val &v){
    access(u); cur.v = v; up(u); }
void set_sval(int u, const SVal &v){
```

```
access(u); cur.sv = v; up(u); }
Val query(int x, int y){
    split(x, y); return o[y].sum; }
SVal subtree(int p, int u){
    chroot(p); access(u); return cur.vir + cur.sv; }
bool connected(int u, int v){
    return find_root(u) == find_root(v); }
void link(int x, int y){
    chroot(x); assert(find_root(y) != x); access(y);
    o[y].vir = o[y].vir + o[x].sub;
    up(o[x].pa = y);
}
void cut(int x, int y){
    split(x, y); assert(o[y].ch[0] == x && !o[x].ch[1]);
    o[y].ch[0] = o[x].pa = 0; up(y); }
#undef cur
#undef lc
#undef rc
};

DirectedMST.h
Description: Finds a minimum spanning tree/arborescence of a directed
graph, given a root node. If no MST exists, returns -1.
Time: O(E log V)
"../data-structures/UnionFindRollback.h" 39e620, 60 lines

struct Edge { int a, b; ll w; };
struct Node {
    Edge key;
    Node *l, *r;
    ll delta;
    void prop(){
        key.w += delta;
        if(l) l->delta += delta;
        if(r) r->delta += delta;
        delta = 0;
    }
    Edge top(){ prop(); return key; }
};
Node *merge(Node *a, Node *b){
    if(!a || !b) return a ? b;
    a->prop(), b->prop();
    if(a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}
void pop(Node*& a){ a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g){
    RollbackUF uf(n);
    vector<Node*> heap(n);
    for(Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
    ll res = 0;
    vi seen(n, -1), path(n, par(n));
    seen[r] = r;
    vector<Edge> Q(n), in(n, {-1, -1}), comp;
    deque<tuple<int, int, vector<Edge>>> cycs;
    rep(s, 0, n){
        int u = s, qi = 0, w;
        while(seen[u] < 0){
            if(!heap[u]) return {-1, {}};
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if(seen[u] == s){
                Node* cyc = 0;
                int end = qi, time = uf.time();
                do cyc = merge(cyc, heap[w = path[--qi]]);
                while(uf.join(u, w));
            }
        }
    }
}
```

```
u = uf.find(u), heap[u] = cyc, seen[u] = -1;
cycs.push_front({u, time, {&Q[qi], &Q[end]}});
}
}
rep(i, 0, qi) in[uf.find(Q[i].b)] = Q[i];
}

for(auto& [u, t, comp] : cycs){ // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for(auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
}
rep(i, 0, n) par[i] = in[i].a;
return {res, par};
}

7.8 Math
7.8.1 Number of Spanning Trees
Create an  $N \times N$  matrix  $\text{mat}$ , and for each edge  $a \rightarrow b \in G$ , do
 $\text{mat}[a][b]--$ ,  $\text{mat}[b][b]++$  (and  $\text{mat}[b][a]--$ ,
 $\text{mat}[a][a]++$  if  $G$  is undirected). Remove the  $i$ th row and
column and take the determinant; this yields the number of
directed spanning trees rooted at  $i$  (if  $G$  is undirected, remove
any row/column).

7.8.2 Erdős–Gallai theorem
A simple graph with node degrees  $d_1 \geq \dots \geq d_n$  exists iff
 $d_1 + \dots + d_n$  is even and for every  $k = 1 \dots n$ ,


$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$


Geometry (8)
8.1 Geometry
GeometryTemplate.cpp
Description: Common 2D geometry primitives for pair-based points, in-
cluding vector arithmetic, orientation, segment tests, line intersection, pro-
jection, and reflection.
Time: O(1) per operation dab500, 57 lines

using ld = double;
using pdd = pair<ld, ld>;
#define X first
#define Y second
const ld eps = 1e-9;

#define TEMP template<class T>
#define ptt pair<T, T>
TEMP ptt operator+(ptt a, ptt b)
{ return {a.X + b.X, a.Y + b.Y}; }
TEMP ptt operator-(ptt a, ptt b)
{ return {a.X - b.X, a.Y - b.Y}; }
TEMP ptt operator*(T i, ptt v)
{ return {i * v.X, i * v.Y}; }
TEMP ptt operator*(ptt v, T i)
{ return {i * v.X, i * v.Y}; }
TEMP pdd operator/(ptt v, ld i)
{ return {v.X / i, v.Y / i}; }
TEMP T dot(ptt a, ptt b)
```

```
{ return a.X * b.X + a.Y * b.Y; }
TEMP T cross(ptt a, ptt b)
{ return a.X * b.Y - a.Y * b.X; }
TEMP T abs2(ptt v)
{ return v.X * v.X + v.Y * v.Y; };
TEMP ld abs(ptt v)
{ return sqrt(abs2(v)); };
int sgn(ll v)
{ return v > 0 ? 1 : (v < 0 ? -1 : 0); }
int sgn(ld v){ return v > eps ? 1 : ( v < -eps ? -1 : 0); }
TEMP int ori(ptt a, ptt b, ptt c)
{ return sgn(cross(b - a, c - a)); }
TEMP bool collinearity(ptt a, ptt b, ptt c)
{ return ori(a, b, c) == 0; }
TEMP bool btw(ptt p, ptt a, ptt b)
{ return collinearity(p, a, b) && sgn(dot(a - p, b - p)) <= 0;
}
```

```
TEMP bool seg_intersect(ptt p1, ptt p2, ptt p3, ptt p4){
    if(btw(p1, p3, p4) || btw(p2, p3, p4) || btw(p3, p1, p2) ||
       btw(p4, p1, p2))
        return true;
    return ori(p1, p2, p3) * ori(p1, p2, p4) < 0 &&
           ori(p3, p4, p1) * ori(p3, p4, p2) < 0;
}
pdd intersect(pdd p1, pdd p2, pdd p3, pdd p4){
    ld a123 = cross(p2 - p1, p3 - p1);
    ld a124 = cross(p2 - p1, p4 - p1);
    return (p4 * a123 - p3 * a124) / (a123 - a124);
}
TEMP ptt perp(ptt p1)
{ return {-p1.Y, p1.X}; }
pdd projection(pdd p1, pdd p2, pdd p3)
{ return p1 + (p2 - p1) * dot(p3 - p1, p2 - p1) / abs2(p2 - p1)
; }
pdd reflection(pdd p1, pdd p2, pdd p3)
{ return p3 + perp(p2 - p1) * cross(p3 - p1, p2 - p1) / abs2(p2
- p1) * 2.0; }
pdd linearTransformation(pdd p0, pdd p1, pdd q0, pdd q1, pdd r)
{
    pdd dp = p1 - p0, dq = q1 - q0, num(cross(dp, dq), dot(dp, dq
));
    return q0 + pdd(cross(r - p0, num), dot(r - p0, num)) / abs2(
dp);
}
// from line p0—p1 to q0—q1, apply to r
```

CycTsearch.cpp
Description: Given a predicate over adjacent values of a cyclically shifted unimodal sequence, returns an index for which pred(x, index) is false for every x.
Time: $\mathcal{O}(\log N)$

```
int cyc_tsearch(int n, auto pred) {
    if (n == 1) return 0;
    int l = 0, r = n; bool rv = pred(1, 0);
    while (r - l > 1) {
        int m = (l + r) / 2;
        if (pred(0, m) ? rv: pred(m, (m + 1) % n)) r = m;
        else l = m;
    }
    return pred(1, r % n) ? l : r % n;
}
```

ConvexHull.cpp
Description: Computes the indices of the strict convex hull of a set of 2D points. Collinear points on hull edges are discarded.
Time: $\mathcal{O}(N \log N)$

```
vector<int> getConvexHull(vector<pdd>& pts){
```

```
vector<int> id(sz(pts));
iota(all(id), 0);
sort(all(id), [&](int x, int y){ return pts[x] < pts[y]; });
vector<int> hull;
for(int tt = 0; tt < 2; tt++){
    int sz = sz(hull);
    for(int j : id){
        pdd p = pts[j];
        while(sz(hull) - sz >= 2 && ori(pts[hull.end()[-2]], pts[
hull.back()], p) <= 0)
            hull.pop_back();
        hull.pb(j);
    }
    hull.pop_back();
    reverse(all(id));
}
return hull;
}
```

PolarAngleComp.cpp
Description: Compares integer vectors by polar angle without trigonometry. Equal directions can be ordered by squared length or reported as equivalent.
Time: $\mathcal{O}(1)$

```
// -1: a // b (if same), 0/1: a < b
int cmp(p11 a, p11 b, bool same = true){
#define is_neg(k) (sgn(k.Y) < 0 || (sgn(k.Y) == 0 && sgn(k.X) <
0))
    int A = is_neg(a), B = is_neg(b);
    if(A != B)
        return A < B;
    if(sgn(cross(a, b)) == 0)
        return same ? abs2(a) < abs2(b) : -1;
    return sgn(cross(a, b)) > 0;
}
```

MinkowskiSum.cpp
Description: Computes the Minkowski sum of two convex polygons by merging their edge directions after rotating both polygons to a canonical starting vertex.
Time: $\mathcal{O}(N + M)$

```
TEMP void reorder_poly(vector<ptt>& pts){
    rotate(pts.begin(), min_element(all(pts),
[&](ptt x, ptt y){
        return x.Y != y.Y ? x.Y < y.Y : x.X < y.X; })),
pts.end());
}
TEMP vector<ptt> minkowski(vector<ptt> P, vector<ptt> Q){
    reorder_poly(P); reorder_poly(Q);
    int psz = P.size(), qsz = Q.size();
    P.pb(P[0]); P.pb(P[1]); Q.pb(Q[0]); Q.pb(Q[1]);
    vector<ptt> ans; int i = 0, j = 0;
    while (i < psz || j < qsz) {
        ans.pb(P[i] + Q[j]);
        int t = sgn(cross(P[i + 1]-P[i], Q[j + 1]-Q[j]));
        if(t >= 0) i++; if(t <= 0) j++;
    }
    return ans;
}
```

CircleConvexIntersection.cpp
Description: Computes the area of intersection between a circle and a polygon whose vertices are given in boundary order.
Time: $\mathcal{O}(N)$

```
double _area(pdd pa, pdd pb, double r){
    if(abs(pa)<abs(pb)) swap(pa, pb);
```

```
if(abs(pb)<eps) return 0;
double S, h, theta;
double a=abs(pb),b=abs(pa),c=abs(pb-pa);
double cosB = dot(pb,pb-pa) / a / c, B = acos(cosB);
double cosC = dot(pa,pb) / a / b, C = acos(cosC);
if(a > r){
    S = (C/2)*r*r;
    h = a*b*sin(C)/c;
    if (h < r && B < PI/2) S -= (acos(h/r)*r*r - h*sqrt(r*r-h*h
));
}
else if(b > r){
    theta = PI - B - asin(sin(B)/r*a);
    S = .5*a*r*sin(theta) + (C-theta)/2*r*r;
}
else S = .5*sin(C)*a*b;
return S;
}
double areaPolyCircle(const vector<pdd> poly,const pdd &o,const
double r){
    double S=0;
    for(int i=0;i<sz(poly);++i)
        S+=_area(poly[i]-o,poly[(i+1)%sz(poly)]-o,r)*ori(o,poly[i],
poly[(i+1)%sz(poly)]);
    return fabs(S);
}
```

CircleCircleIntersection.cpp
Description: Computes the intersection points of two non-concentric circles. Returns false when the circles are disjoint or one lies strictly inside the other.
Time: $\mathcal{O}(1)$

```
bool CCinter(Cir &a, Cir &b, pdd &p1, pdd &p2) {
    pdd o1 = a.O, o2 = b.O;
    double r1 = a.R, r2 = b.R, d2 = abs2(o1 - o2), d = sqrt(d2);
    if(d < max(r1, r2) - min(r1, r2) || d > r1 + r2) return 0;
    pdd u = (o1 + o2) * 0.5 + (o1 - o2) * ((r2 * r2 - r1 * r1) /
(2 * d2));
    double A = sqrt((r1 + r2 + d) * (r1 - r2 + d) * (r1 + r2 - d)
* (-r1 + r2 + d));
    pdd v = pdd(o1.Y - o2.Y, -o1.X + o2.X) * A / (2 * d2);
    p1 = u + v, p2 = u - v;
    return 1;
}
```

CircleCircleTangent.cpp
Description: Returns the common tangent lines of two distinct circles. Use sign1 = 1 for outer tangents and sign1 = -1 for inner tangents.
Time: $\mathcal{O}(1)$

```
vector<Line> CCTang( const Cir& c1 , const Cir& c2 , int sign1
){
    vector<Line> ret;
    double d_sq = abs2( c1.O - c2.O );
    if (sgn(d_sq) == 0) return ret;
    double d = sqrt(d_sq);
    pdd v = (c2.O - c1.O) / d;
    double c = (c1.R - sign1 * c2.R) / d; // cos t
    if (c * c > 1) return ret;
    double h = sqrt(max( 0.0, 1.0 - c * c )); // sin t
    for (int sign2 = 1; sign2 >= -1; sign2 -= 2) {
        pdd n = pdd(v.X * c - sign2 * h * v.Y,
v.Y * c + sign2 * h * v.X);
        pdd p1 = c1.O + n * c1.R;
        pdd p2 = c2.O + n * (c2.R * sign1);
        if (sgn(p1.X - p2.X) == 0 and
sgn(p1.Y - p2.Y) == 0)
            p2 = p1 + perp(c2.O - c1.O);
```

```
    ret.pb(Line(p1, p2));
}
return ret;
}
```

ConvexLineIntersection.cpp

Description: Finds the sides or vertex where an infinite directed line intersects a convex polygon. Returns -1, -1 when there is no intersection.
Time: $\mathcal{O}(\log N)$

7d8485, 32 lines

```
int TangentDir(vector<pll> &C, pll dir) {
    return cyc_tsearch(sz(C), [&](int a, int b) {
        return cross(dir, C[a]) > cross(dir, C[b]);
    });
}
#define cmpL(i) sgn(cross(C[i] - a, b - a))
pii lineHull(pll a, pll b, vector<pll> &C) {
    int A = TangentDir(C, a - b);
    int B = TangentDir(C, b - a);
    int n = sz(C);
    if (cmpL(A) < 0 || cmpL(B) > 0)
        return pii(-1, -1); // no collision
    auto gao = [&](int l, int r) {
        for (int t = l; (l + 1) % n != r; ) {
            int m = ((l + r + (l < r ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(t) ? l : r) = m;
        }
        return (l + !cmpL(r)) % n;
    };
    pii res = pii(gao(B, A), gao(A, B)); // (i, j)
    if (res.X == res.Y) // touching the corner i
        return pii(res.X, -1);
    if (!cmpL(res.X) && !cmpL(res.Y)) // along side i, i+1
        switch ((res.X - res.Y + n + 1) % n) {
            case 0: return pii(res.X, res.X);
            case 2: return pii(res.Y, res.Y);
        }
    /* crossing sides (i, i+1) and (j, j+1)
    crossing corner i is treated as side (i, i+1)
    returned in the same order as the line hits the convex */
    return res;
} // convex cut: (r, l)
```

CircleLineIntersection.cpp

Description: Returns the zero, one, or two intersection points between a circle and the infinite line through points a and b.
Time: $\mathcal{O}(1)$

9183db, 8 lines

```
vector<pdd> circleLineIntersection(pdd c, double r, pdd a, pdd
    b) {
    pdd p = a + (b - a) * dot(c - a, b - a) / abs2(b - a);
    double s = cross(b - a, c - a), h2 = r * r - s * s / abs2(b -
        a);
    if (sgn(h2) < 0) return {};
    if (sgn(h2) == 0) return {p};
    pdd h = (b - a) / abs(b - a) * sqrt(h2);
    return {p - h, p + h};
}
```

PointInCircle.cpp

Description: Uses an exact determinant to test whether an integer point lies strictly inside the circumcircle of a counter-clockwise integer triangle.
Time: $\mathcal{O}(1)$

ecf954, 7 lines

```
// return q's relation with circumcircle of tri(p[0],p[1],p[2])
bool in_cc(const array<pll, 3> &p, pll q) {
    __int128 det = 0;
    for (int i = 0; i < 3; ++i)
```

```
        det += __int128(abs2(p[i]) - abs2(q)) * cross(p[(i + 1) %
            3] - q, p[(i + 2) % 3] - q);
    return det > 0; // in: >0, on: =0, out: <0
}
```

PointInConvex.cpp

Description: Tests whether a point lies inside a convex polygon in logarithmic time. The strict flag controls whether boundary points are accepted.
Time: $\mathcal{O}(\log N)$

fe4934, 13 lines

```
bool PointInConvex(const vector<pll> &C, pll p, bool strict =
    true) {
    int a = 1, b = sz(C) - 1, r = !strict;
    if (sz(C) == 0) return false;
    if (sz(C) < 3) return r && btw(p, C[0], C.back());
    if (ori(C[0], C[a], C[b]) > 0) swap(a, b);
    if (ori(C[0], C[a], p) >= r || ori(C[0], C[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (ori(C[0], C[c], p) > 0 ? b : a) = c;
    }
    return ori(C[a], C[b], p) < r;
}
```

HalfPlaneIntersection.cpp

Description: Intersects directed half-planes and returns the boundary lines of the resulting convex region. Assumes the intersection is non-empty and checks strict inclusion.
Time: $\mathcal{O}(N \log N)$

3d7062, 32 lines

```
pll area_pair(Line a, Line b)
{ return pll(cross(a.Y - a.X, b.X - a.X), cross(a.Y - a.X, b.Y
    - a.X)); }
bool isin(Line l0, Line l1, Line l2) {
    // Check inter(l1, l2) strictly in l0
    auto [a02X, a02Y] = area_pair(l0, l2);
    auto [a12X, a12Y] = area_pair(l1, l2);
    if (a12X - a12Y < 0) a12X *= -1, a12Y *= -1;
    return (__int128) a02Y * a12X - (__int128) a02X * a12Y > 0;
}
/* Having solution, check size > 2 */
/* — Line.X — Line.Y — */
vector<Line> halfPlaneInter(vector<Line> arr) {
    sort(all(arr), [&](Line a, Line b) -> int {
        if (cmp(a.Y - a.X, b.Y - b.X, 0) != -1)
            return cmp(a.Y - a.X, b.Y - b.X, 0);
        return ori(a.X, a.Y, b.Y) < 0;
    });
    deque<Line> dq(l, arr[0]);
    auto pop_back = [&](int t, Line p) {
        while (sz(dq) >= t && !isin(p, dq[sz(dq) - 2], dq.back()))
            dq.pop_back();
    };
    auto pop_front = [&](int t, Line p) {
        while (sz(dq) >= t && !isin(p, dq[0], dq[l]))
            dq.pop_front();
    };
    for (auto p : arr)
        if (cmp(dq.back().Y - dq.back().X, p.Y - p.X, 0) != -1)
            pop_back(2, p), pop_front(2, p), dq.pb(p);
    pop_back(3, dq[0]), pop_front(3, dq.back());
    return vector<Line>(all(dq));
}
```

HPIGeneralLine.cpp

Description: Exact integer-line helpers for half-plane intersection. Lines are stored as $ax + by + c \leq 0$, while intersections are returned as floating-point points.

ba52e5, 24 lines

```
Time:  $\mathcal{O}(1)$  per operation

// replace corresponding functions in vector HPI
using i128 = __int128;
struct LN {
    ll a, b, c; // ax + by + c <= 0
    pll dir() const { return pll(a, b); }
    LN(ll ta, ll tb, ll tc) : a(ta), b(tb), c(tc) {}
    LN(pll S, pll T) : a((T-S).Y), b(-(T-S).X), c(cross(T,S)) {}
};
pdd intersect(LN A, LN B) {
    double c = cross(A.dir(), B.dir());
    i128 a = i128(A.c) * B.a - i128(B.c) * A.a;
    i128 b = i128(A.c) * B.b - i128(B.c) * A.b;
    return pdd(-b / c, a / c);
}
bool isin(LN l, LN A, LN B) {
    i128 c = cross(A.dir(), B.dir());
    i128 a = i128(A.c) * B.a - i128(B.c) * A.a;
    i128 b = i128(A.c) * B.b - i128(B.c) * A.b;
    return sgn(a * l.b - b * l.a + c * l.c) * sgn(c) < 0;
}
int cmp(LN a, LN b) {
    if (int c = cmp(a.dir(), b.dir(), false); c != -1) return c;
    return i128(abs(b.a) + abs(b.b)) * a.c > i128(abs(a.a) + abs(
        a.b)) * b.c;
}


```

MinimumEnclosingCircle.cpp

Description: Randomized incremental algorithm for the minimum enclosing circle of a non-empty point set. Requires a circumcenter routine for three boundary points.
Time: $\mathcal{O}(N)$ expected

88c1e4, 21 lines

```
using ld = long double;
pair<pdd, ld> circumcenter(pdd a, pdd b, pdd c);
pair<pdd, ld> MinimumEnclosingCircle(vector<pdd> &pts){
    random_shuffle(all(pts));
    pdd c = pts[0];
    ld r = 0;
    for(int i = 1; i < sz(pts); i++){
        if(abs(pts[i] - c) <= r) continue;
        c = pts[i]; r = 0;
        for(int j = 0; j < i; j++){
            if(abs(pts[j] - c) <= r) continue;
            c = (pts[i] + pts[j]) / 2;
            r = abs(pts[i] - c);
            for(int k = 0; k < j; k++){
                if(abs(pts[k] - c) > r)
                    tie(c, r) = circumcenter(pts[i], pts[j], pts[k]);
            }
        }
    }
    return {c, r};
}
```

3DPoint.cpp

Description: Basic 3D point and vector operations, including dot and cross products, signed volume, spherical angles, projection onto a plane, and rotation around an axis.
Time: $\mathcal{O}(1)$ per operation

badbbd, 46 lines

```
struct Point {
    double x, y, z;
    Point(double _x = 0, double _y = 0, double _z = 0): x(_x), y(
        _y), z(_z){}
    Point(pdd p) { x = p.X, y = p.Y, z = abs2(p); }
};
Point operator-(Point p1, Point p2)
{ return Point(p1.x - p2.x, p1.y - p2.y, p1.z - p2.z); }
```

```
Point operator+(Point p1, Point p2)
{ return Point(p1.x + p2.x, p1.y + p2.y, p1.z + p2.z); }
Point operator*(Point p1, double v)
{ return Point(p1.x * v, p1.y * v, p1.z * v); }
Point operator/(Point p1, double v)
{ return Point(p1.x / v, p1.y / v, p1.z / v); }
Point cross(Point p1, Point p2)
{ return Point(p1.y * p2.z - p1.z * p2.y, p1.z * p2.x - p1.x *
    p2.z, p1.x * p2.y - p1.y * p2.x); }
double dot(Point p1, Point p2)
{ return p1.x * p2.x + p1.y * p2.y + p1.z * p2.z; }
double abs(Point a)
{ return sqrt(dot(a, a)); }
Point cross3(Point a, Point b, Point c)
{ return cross(b - a, c - a); }
double area(Point a, Point b, Point c)
{ return abs(cross3(a, b, c)); }
double volume(Point a, Point b, Point c, Point d)
{ return dot(cross3(a, b, c), d - a); }
//Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
double phi(Point p) { return atan2(p.y, p.x); }
//Zenith angle (latitude) to the z-axis in interval [0, pi]
double theta(Point p) { return atan2(sqrt(p.x * p.x + p.y * p.y
    ), p.z); }
Point masscenter(Point a, Point b, Point c, Point d)
{ return (a + b + c + d) / 4; }
pdd proj(Point a, Point b, Point c, Point u) {
// proj. u to the plane of a, b, and c
    Point e1 = b - a;
    Point e2 = c - a;
    e1 = e1 / abs(e1);
    e2 = e2 - e1 * dot(e2, e1);
    e2 = e2 / abs(e2);
    Point p = u - a;
    return pdd(dot(p, e1), dot(p, e2));
}
Point rotate_around(Point p, double angle, Point axis) {
    double s = sin(angle), c = cos(angle);
    Point u = axis / abs(axis);
    return u * dot(u, p) * (1 - c) + p * c + cross(u, p) * s;
}
```

ConvexHull3D.cpp

Description: Incrementally constructs the triangular faces of a 3D convex hull and provides its coplanar face count, volume, and point-to-face distance.
Time: $\mathcal{O}(N^2)$ construction

651397, 65 lines

```
struct convex_hull_3D {
struct Face {
    int a, b, c;
    Face(int ta, int tb, int tc): a(ta), b(tb), c(tc) {}
}; // return the faces with pt indexes
vector<Face> res;
vector<Point> P;
convex_hull_3D(const vector<Point> &_P): res(), P(_P) {
// all points coplanar case will WA  $\mathcal{O}(n^2)$ 
    int n = sz(P);
    if (n <= 2) return; // be careful about edge case
    // ensure first 4 points are not coplanar
    swap(P[1], *find_if(all(P), [&](auto p) { return sgn(abs2(P
        [0] - p)) != 0; }));
    swap(P[2], *find_if(all(P), [&](auto p) { return sgn(abs2(
        cross3(p, P[0], P[1])) != 0; }));
    swap(P[3], *find_if(all(P), [&](auto p) { return sgn(volume(P
        [0], P[1], P[2], p)) != 0; }));
    vector<vector<int>>> flag(n, vector<int>(n));
    res.emplace_back(0, 1, 2); res.emplace_back(2, 1, 0);
    for (int i = 3; i < n; ++i) {
        vector<Face> next;
```

```
        for (auto f : res) {
            int d = sgn(volume(P[f.a], P[f.b], P[f.c], P[i]));
            if (d <= 0) next.pb(f);
            int ff = (d > 0) - (d < 0);
            flag[f.a][f.b] = flag[f.b][f.c] = flag[f.c][f.a] = ff;
        }
        for (auto f : res) {
            auto F = [&](int x, int y) {
                if (flag[x][y] > 0 && flag[y][x] <= 0)
                    next.emplace_back(x, y, i);
            };
            F(f.a, f.b); F(f.b, f.c); F(f.c, f.a);
        }
        res = next;
    }
}
bool same(Face s, Face t) {
    if (sgn(volume(P[s.a], P[s.b], P[s.c], P[t.a])) != 0) return
        0;
    if (sgn(volume(P[s.a], P[s.b], P[s.c], P[t.b])) != 0) return
        0;
    if (sgn(volume(P[s.a], P[s.b], P[s.c], P[t.c])) != 0) return
        0;
    return 1;
}
int polygon_face_num() {
    int ans = 0;
    for (int i = 0; i < sz(res); ++i)
        ans += none_of(res.begin(), res.begin() + i, [&](Face g) {
            return same(res[i], g); });
    return ans;
}
double get_volume() {
    double ans = 0;
    for (auto f : res)
        ans += volume(Point(0, 0, 0), P[f.a], P[f.b], P[f.c]);
    return fabs(ans / 6);
}
double get_dis(Point p, Face f) {
    Point p1 = P[f.a], p2 = P[f.b], p3 = P[f.c];
    double a = (p2.y - p1.y) * (p3.z - p1.z) - (p2.z - p1.z) * (
        p3.y - p1.y);
    double b = (p2.z - p1.z) * (p3.x - p1.x) - (p2.x - p1.x) * (
        p3.z - p1.z);
    double c = (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (
        p3.x - p1.x);
    double d = 0 - (a * p1.x + b * p1.y + c * p1.z);
    return fabs(a * p.x + b * p.y + c * p.z + d) / sqrt(a * a + b
        * b + c * c);
}
};
// n^2 delaunay: facets with negative z normal of
// convexhull of (x, y, x^2 + y^2), use a pseudo-point
// (0, 0, inf) to avoid degenerate case
```

DelaunayTriangulation.cpp

Description: Constructs an undirected Delaunay triangulation of 2D inter-ger points by divide and conquer. No point may lie strictly inside a resulting circumcircle.
Time: $\mathcal{O}(N^2)$

1131c7, 60 lines

```
/* Delaunay Triangulation:
    Given a sets of points on 2D plane, find a
    triangulation such that no points will strictly
    inside circumcircle of any triangle. */
struct Edge {
    int id; // oidx[id]
    list<Edge>::iterator twin;
    Edge(int _id = 0):id(_id) {}
};
```

```
};
struct Delaunay { // 0-base
    int n;
    vector<int> oidx;
    vector<list<Edge>> head; // result udir. graph
    vector<pll> p;
    Delaunay(int _n, vector<pll> _p): n(_n), oidx(n), head(n), p(
        n) {
        iota(all(oidx), 0);
        for (int i = 0; i < n; ++i) head[i].clear();
        sort(all(oidx), [&](int a, int b)
            { return _p[a] < _p[b]; });
        for (int i = 0; i < n; ++i) p[i] = _p[oidx[i]];
        divide(0, n - 1);
    }
    void addEdge(int u, int v) {
        head[u].push_front(Edge(v));
        head[v].push_front(Edge(u));
        head[u].begin()->twin = head[v].begin();
        head[v].begin()->twin = head[u].begin();
    }
    void divide(int l, int r) {
        if (l == r) return;
        if (l + 1 == r) return addEdge(l, l + 1);
        int mid = (l + r) >> 1, nw[2] = {l, r};
        divide(l, mid), divide(mid + 1, r);
        auto gao = [&](int t) {
            pll pt[2] = {p[nw[0]], p[nw[1]]};
            for (auto it : head[nw[t]]) {
                int v = ori(pt[1], pt[0], p[it.id]);
                if (v > 0 || (v == 0 && abs2(pt[t ^ 1] - p[it.id]) <
                    abs2(pt[1] - pt[0])))
                    return nw[t] = it.id, true;
            }
            return false;
        };
        while (gao(0) || gao(1));
        addEdge(nw[0], nw[1]); // add tangent
        while (true) {
            pll pt[2] = {p[nw[0]], p[nw[1]]};
            int ch = -1, sd = 0;
            for (int t = 0; t < 2; ++t)
                for (auto it : head[nw[t]])
                    if (ori(pt[0], pt[1], p[it.id]) > 0 && (ch == -1 ||
                        in_cc({pt[0], pt[1], p[ch]}, p[it.id])))
                        ch = it.id, sd = t;
            if (ch == -1) break; // upper common tangent
            for (auto it = head[nw[sd]].begin(); it != head[nw[sd]].
                end(); )
                if (seg_strict_intersect(pt[sd], p[it->id], pt[sd ^ 1],
                    p[ch]))
                    head[it->id].erase(it->twin), head[nw[sd]].erase(it
                        ++);
            else ++it;
            nw[sd] = ch, addEdge(nw[0], nw[1]);
        }
    }
};
```

VoronoiDiagram.cpp

Description: Builds the perpendicular-bisector half-planes of every Voronoi cell from a Delaunay triangulation. Run half-plane intersection to obtain the actual cells.
Time: $\mathcal{O}(N^2)$ including Delaunay construction

e4f408, 12 lines

```
// all coord. is even, you may want to call halfPlaneInter
    after then
vector<vector<Line>>> vec;
void build_voronoi_line(int n, vector<pll> &pts) {
```

```

Delaunay tool(n, pts); // Delaunay
vec.clear(), vec.resize(n);
for (int i = 0; i < n; ++i)
    for (auto e : tool.head[i]) {
        int u = tool.oidx[i], v = tool.oidx[e.id];
        pll m = (pts[v] + pts[u]) / 2LL, d = perp(pts[v] - pts[u]
        );
        vec[u].pb(Line(m, m + d));
    }
}
```

PolyUnion.cpp

Description: Computes the area of the union of counter-clockwise integer polygons by splitting each edge into covered and uncovered parameter intervals.

Time: $\mathcal{O}(E^2 \log E)$, where E is the total number of polygon edges.

```

ld rat(pll a, pll b) {
    return sgn(b.X) ? (ld)a.X / b.X : (ld)a.Y / b.Y;
} // all poly. should be ccw
ld polyUnion(vector<vector<pll>> &poly) {
    ld res = 0;
    for (auto &p : poly)
        for (int a = 0; a < sz(p); ++a) {
            pll A = p[a], B = p[(a + 1) % sz(p)];
            vector<pair<ld, int>> segs = {{0, 0}, {1, 0}};
            for (auto &q : poly) {
                if (&p == &q) continue;
                for (int b = 0; b < sz(q); ++b) {
                    pll C = q[b], D = q[(b + 1) % sz(q)];
                    int sc = ori(A, B, C), sd = ori(A, B, D);
                    if (sc != sd && min(sc, sd) < 0) {
                        ld sa = cross(D - C, A - C), sb = cross(D - C, B - C);
                        segs.pb(sa / (sa - sb), sgn(sc - sd));
                    }
                    if (!sc && !sd && &q < &p && sgn(dot(B - A, D - C)) > 0) {
                        segs.pb(rat(C - A, B - A), 1);
                        segs.pb(rat(D - A, B - A), -1);
                    }
                }
            }
            sort(all(segs));
            for (auto &s : segs) s.X = clamp(s.X, 0.0, 1.0);
            ld sum = 0;
            int cnt = segs[0].second;
            for (int j = 1; j < sz(segs); ++j) {
                if (!cnt) sum += segs[j].X - segs[j - 1].X;
                cnt += segs[j].Y;
            }
            res += cross(A, B) * sum;
        }
    return res / 2;
}
```

TangentPointToHull.cpp

Description: Finds one contact vertex for each tangent from a point strictly outside a convex polygon. The polygon must be stored in cyclic order.

Time: $\mathcal{O}(\log N)$

```

/* The point should be strictly out of hull
   return arbitrary point on the tangent line */
pii get_tangent(vector<pll> &C, pll p) {
    auto gao = [&](int s) {
        return cyc_tsearch(sz(C), [&](int x, int y)
        { return ori(p, C[x], C[y]) == s; });
    };
    return pii(gao(1), gao(-1));
}
```

```

} // return (a, b), ori(p, C[a], C[b]) >= 0
```

Heart.cpp

Description: Computes the circumcenter, incenter, centroid, and orthocenter of a non-degenerate triangle.

```

Time:  $\mathcal{O}(1)$ 
082d19, 18 lines

pdd circenter(pdd p0, pdd p1, pdd p2) { // SCOPE HASH
    p1 = p1 - p0, p2 = p2 - p0; // radius = abs(center)
    double x1 = p1.X, y1 = p1.Y, x2 = p2.X, y2 = p2.Y;
    double m = 2. * (x1 * y2 - y1 * x2);
    pdd center;
    center.X = (x1 * x1 * y2 - x2 * x2 * y1 + y1 * y2 * (y1 - y2)
    ) / m;
    center.Y = (x1 * x2 * (x2 - x1) - y1 * y1 * x2 + x1 * y2 * y2
    ) / m;
    return center + p0;
}

pdd incenter(pdd p1, pdd p2, pdd p3) { // radius = area / s * 2
    double a = abs(p2 - p3), b = abs(p1 - p3), c = abs(p1 - p2);
    double s = a + b + c;
    return (a * p1 + b * p2 + c * p3) / s;
}

pdd masscenter(pdd p1, pdd p2, pdd p3)
{ return (p1 + p2 + p3) / 3; }

pdd orthcenter(pdd p1, pdd p2, pdd p3)
{ return masscenter(p1, p2, p3) * 3 - circenter(p1, p2, p3) * 2; }
```

RotatingSweepLine.cpp

Description: Rotates a sweep direction through all pairwise point directions while maintaining the order of points and grouping simultaneous collinear events.

```

Time:  $\mathcal{O}(N^2 \log N)$ 
af24d8, 31 lines

struct Event {
    pll d; int u, v;
    bool operator<(const Event &b) const {
        int ret = cmp(d, b.d, false);
        return ret == -1 ? false : ret; } // no tie-break
};

void rotatingSweepLine(const vector<pll> &p) {
    const int n = sz(p);
    vector<Event> e; e.reserve(n * (n - 1));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) // pos[i] < pos[j] when the
            // event occurs
            if (i != j) e.pb(p[j] - p[i], i, j);
    sort(all(e));
    vector<int> ord(n), pos(n);
    iota(all(ord), 0);
    sort(all(ord), [&](int i, int j) { // initial order
        return p[i].Y != p[j].Y ? p[i].Y < p[j].Y : p[i].X < p[j]
        ].X; });
    for (int i = 0; i < n; i++) pos[ord[i]] = i;
    // initialize
    for (int i = 0, j = 0; i < sz(e); i = j) {
        // do something
        vector<pii> tmp;
        for (; j < sz(e) && !(e[i] < e[j]); j++)
            tmp.pb(pii(e[j].u, e[j].v));
        sort(all(tmp), [&](pii x, pii y){
            return pii(pos[x.F], pos[x.S]) < pii(pos[y.F], pos[y.S]
            ); });
        for (auto [x, y] : tmp) // pos[x] + 1 == pos[y]
            tie(ord[pos[x]], ord[pos[y]], pos[x], pos[y]) =
            make_tuple(ord[pos[y]], ord[pos[x]], pos[y], pos[x]);
    }
}
```

VectorInPoly.cpp

Description: Tests whether a directed vector from a polygon vertex lies inside the local angle formed by the previous and next vertices, with optional strict boundaries.

```

Time:  $\mathcal{O}(1)$ 
c6d0fa, 10 lines

// ori(a, b, c) >= 0, valid: "strict" angle from a-b to a-c
bool btwangle(pll a, pll b, pll c, pll p, int strict) {
    return ori(a, b, p) >= strict && ori(a, p, c) >= strict;
}

// whether vector{cur, p} in counter-clockwise order prv, cur,
//   nxt
bool inside(pll prv, pll cur, pll nxt, pll p, int strict) {
    if (ori(cur, nxt, prv) >= 0)
        return btwangle(cur, nxt, prv, p, strict);
    return !btwangle(cur, prv, nxt, p, !strict);
}
```

ConvexHullDP.cpp

Description: Orders directed point pairs by angle and provides an $\mathcal{O}(N^3)$ transition skeleton for dynamic programming over convex polygons.

```

Time:  $\mathcal{O}(N^3)$ 
ec5414, 33 lines

sort(all(pts), [&](pll x, pll y) {
    return x.Y != y.Y ? x.Y < y.Y : x.X < y.X;
});
auto getvec = [&](pii x) { return pts[x.S] - pts[x.F]; };
vector<pii> trans;
for (int j = 0; j < n; j++)
    for (int k = 0; k < n; k++)
        if (j != k) trans.pb(pii(j, k));
sort(all(trans), [&](pii x, pii y) -> bool{
    int tmp = cmp(getvec(x), getvec(y), false);
    if (tmp != -1) return tmp;
    pll v = getvec(x);
    return dot(v, pts[x.F]) > dot(v, pts[y.F]);
});
// DP for convex hull vertices (no points on edges)
auto solve = [&](int bottom) { //  $\mathcal{O}(n^3)$ 
    // vector<ll> dp(n);
    for (int j = bottom + 1; j < n; j++) {
        // check whether bottom -> j is legal
        // init trans -> j
    }
    for (auto [i, j] : trans) {
        if (i <= bottom || j <= bottom ||
            ori(pts[bottom], pts[i], pts[j]) <= 0) continue;
        // check whether i -> j is legal
        // normal trans i -> j
    }
    for (int j = bottom + 1; j < n; j++) {
        // check whether j -> bottom is legal
        // end trans j ->
    }
};
for(int i = 0; i < n; i++) solve(i);
```

CalcPointsInTriangle.cpp

Description: Precomputes point counts above every directed pair and answers the number of input points strictly inside a triangle in constant time.

Time: $\mathcal{O}(N^3)$ preprocessing and $\mathcal{O}(1)$ per query

```

// all points are distinct
// cnt[i][j] = # of point k s.t. strictly above ij, and i < k <
//   j
// cnt2[i][j] = # of points k s.t. strictly in ij
// preprocess space:  $\mathcal{O}(n^2)$ , time:  $\mathcal{O}(n^3)$ , query time:  $\mathcal{O}(1)$ 
vector cnt(n, vector<int>(n)), cnt2(n, vector<int>(n));
for (int i = 0; i < n; i++)
```

```
for (int j = 0; j < n; j++){
    if (pts[i] >= pts[j]) continue;
    for (int k = 0; k < n; k++) {
        if (pts[i] < pts[k] && pts[k] < pts[j]) {
            int tmp = ori(pts[i], pts[j], pts[k]);
            if (tmp > 0) cnt[i][j]++; // only for i < j
            else if (tmp == 0) cnt2[i][j]++, cnt2[j][i]++;
        }
    }
}
auto calc_tri = [&](array<int, 3> arr) { // strictly inside
    sort(all(arr), [&](int x, int y){ return pts[x] < pts[y]; });
    auto [x, y, z] = arr;
    int tmp = ori(pts[x], pts[y], pts[z]);
    if (tmp == 0) return 0;
    else if (tmp < 0)
        return cnt[x][z] - cnt[x][y] - cnt[y][z] - cnt2[x][y] - cnt2[y][z] - 1;
    else return cnt[x][y] + cnt[y][z] - cnt[x][z] - cnt2[x][z];
};
```

Strings (9)

Hashing.h
Description: Self-explanatory methods for string hashing. 3fdef6, 44 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
typedef uint64_t ull;
struct H {
    ull x; H(ull x=0) : x(x){}
    H operator+(H o){ return x + o.x + (x + o.x < x); }
    H operator-(H o){ return *this + ~o.x; }
    H operator*(H o){ auto m = (__uint128_t)x * o.x;
        return H((ull)m) + (ull)(m >> 64); }
    ull get() const { return x + !~x; }
    bool operator==(H o) const { return get() == o.get(); }
    bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (1ll)1e11+3; // (order ~ 3e9; random also ok)
```

```
struct HashInterval {
    vector<H> ha, pw;
    HashInterval(string& str) : ha(sz(str)+1), pw(ha){
        pw[0] = 1;
        rep(i, 0, sz(str))
            ha[i+1] = ha[i] * C + str[i],
            pw[i+1] = pw[i] * C;
    }
    H hashInterval(int a, int b){ // hash [a, b)
        return ha[b] - ha[a] * pw[b - a];
    }
};
```

```
H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}
// PART HASH = d099fb
```

```
vector<H> getHashes(string& str, int length){
    if(sz(str) < length) return {};
    H h = 0, pw = 1;
    rep(i, 0, length)
        h = h * C + str[i], pw = pw * C;
    vector<H> ret = {h};
    rep(i, length, sz(str)){
        ret.push_back(h = h * C + str[i] - pw * str[i-length]);
    }
```

```
}
return ret;
} // FULL HASH = 3fdef6
```

KMP.h
Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.
Time: $\mathcal{O}(n)$ d4375c, 16 lines

```
vi pi(const string& s){
    vi p(sz(s));
    rep(i, 1, sz(s)){
        int g = p[i-1];
        while(g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}
```

```
vi match(const string& s, const string& pat){
    vi p = pi(pat + '\0' + s), res;
    rep(i, sz(p)-sz(s), sz(p))
        if(p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}
```

Zfunc.h
Description: z[i] computes the length of the longest common prefix of s[i] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$ ee09e2, 12 lines

```
vi Z(const string& S){
    vi z(sz(S));
    int l = -1, r = -1;
    rep(i, 1, sz(S)){
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while(i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if(i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
```

Manacher.h
Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).
Time: $\mathcal{O}(N)$ e7ad79, 13 lines

```
array<vi, 2> manacher(const string& s){
    int n = sz(s);
    array<vi, 2> p = {vi(n+1), vi(n)};
    rep(z, 0, 2) for(int i=0, l=0, r=0; i < n; i++){
        int t = r-i+!z;
        if(i<r) p[z][i] = min(t, p[z][l+t]);
        int L = i-p[z][i], R = i+p[z][i]-!z;
        while(L>=1 && R+1<n && s[L-1] == s[R+1])
            p[z][i]++, L--, R++;
        if(R>r) l=L, r=R;
    }
    return p;
}
```

MinRotation.h
Description: Finds the lexicographically smallest rotation of a string.
Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());
Time: $\mathcal{O}(N)$ 67652e, 8 lines

```
int minRotation(string s){
    int a=0, N=sz(s); s += s;
    rep(b, 0, N) rep(k, 0, N){
        if(a+k == b || s[a+k] < s[b+k]){b += max<int>(0, k-1);
            break;}
        if(s[a+k] > s[b+k]){a = b; break; }
    }
    return a;
}
```

SuffixArray.h
Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size n + 1, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any nul chars.
Time: $\mathcal{O}(n \log n)$ ed2029, 26 lines

```
struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string s, int lim=256){ // or vector<int>
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vi x(n), y(n), ws(max(n, lim));
        rep(i, 0, n) x[i] = (unsigned char)s[i];
        sa = lcp = y, iota(all(sa), 0);
        for(int j = 0, p = 0; p < n; j = max<int>(1, j * 2), lim = p){
            p = j, iota(all(y), n - j);
            rep(i, 0, n) if(sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i, 0, n) ws[x[i]]++;
            rep(i, 1, lim) ws[i] += ws[i - 1];
            for(int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i, 1, n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        for(int i = 0, j; i < n - 1; i++){
            if(k) k--;
            j = sa[x[i] - 1];
            while(s[i + k] == s[j + k]) k++;
            lcp[x[i]] = k;
        }
    }
};
```

SuffixArraySAIS.h
Description: Linear suffix array by SA-IS. Includes the empty suffix.
Time: $\mathcal{O}(n)$ e66dbd, 66 lines

```
struct SAIS {
    vi sa, lcp;
    vi build(const vi &s, int lim){
        int n = sz(s);
        if(n == 1) return {0};
        vi ret(n, -1), ls(n), sumL(lim + 1), sumS(lim + 1);
        for(int i = n - 2; i >= 0; i--){
            ls[i] = s[i] == s[i + 1] ? ls[i + 1] : s[i] < s[i + 1];
        }
        rep(i, 0, n) (ls[i] ? sumL[s[i] + 1] : sumS[s[i]])++;
        rep(i, 0, lim + 1){
            sumS[i] += sumL[i];
            if(i < lim) sumL[i + 1] += sumS[i];
        }
        auto induce = [&](const vi &lms){
            fill(all(ret), -1);
            vi buf = sumS;
            for(int x : lms) ret[buf[s[x]]++] = x;
            buf = sumL, ret[buf[s[n - 1]]++] = n - 1;
        }
```

```

rep(i, 0, n){
    int x = ret[i] - 1;
    if(x >= 0 && !ls[x]) ret[buf[s[x]]++] = x;
}
buf = sumL;
for(int i = n; i--;){
    int x = ret[i] - 1;
    if(x >= 0 && ls[x]) ret[--buf[s[x] + 1]] = x;
}
};
vi id(n + 1, -1), lms;
rep(i, 1, n) if(!ls[i - 1] && ls[i]) id[i] = sz(lms), lms.pb(i);
induce(lms);
if(sz(lms)){
    vi ord;
    for(int x : ret) if(id[x] != -1) ord.pb(x);
    vi rec(sz(lms)); int names = 0;
    rec[id[ord[0]]] = 0;
    rep(i, 1, sz(lms)){
        int l = ord[i - 1], r = ord[i];
        int el = id[l] + 1 < sz(lms) ? lms[id[l] + 1] : n;
        int er = id[r] + 1 < sz(lms) ? lms[id[r] + 1] : n;
        bool same = el - 1 == er - r;
        while(same && l < el && s[l] == s[r]) l++, r++;
        if(l == n || s[l] != s[r]) same = false;
        rec[id[ord[i]]] = same ? names : ++names;
    }
    vi recSa = build(rec, names);
    rep(i, 0, sz(lms)) ord[i] = lms[recSa[i]];
    induce(ord);
}
return ret;
}
SAIS(string s){
    int n = sz(s), k = 0;
    vi a(n + 1), rk(n + 1);
    rep(i, 0, n) a[i] = (unsigned char)s[i];
    sa = build(a, 255), lcp.assign(n + 1, 0);
    rep(i, 0, n + 1) rk[sa[i]] = i;
    rep(i, 0, n){
        if(k) k--;
        int j = sa[rk[i] - 1];
        while(a[i + k] == a[j + k]) k++;
        lcp[rk[i]] = k;
    }
}
};

```

SuffixTree.h

Description: Ukkonen suffix tree for lowercase strings. Nodes store [l, r].
Time: $\mathcal{O}(28N)$

a14fe6, 57 lines

```

struct SuffixTree {
    enum { ALPHA = 28 };
    int toi(char c){ return c - 'a'; }
    string a;
    vector<array<int, ALPHA>> t;
    vi l, r, p, s;
    int v = 0, q = 0, m = 2;
    void ukkadd(int i, int c){ suff:
        if(r[v] <= q){
            if(t[v][c] == -1){
                t[v][c] = m; l[m] = i; p[m++] = v; v = s[v]; q = r[v];
                goto suff;
            }
            v = t[v][c]; q = l[v];
        }
    }
    if(q == -1 || c == toi(a[q])) q++;
}

```

```

    else{
        l[m+1] = i; p[m+1] = m; l[m] = l[v]; r[m] = q;
        p[m] = p[v]; t[m][c] = m+1; t[m][toi(a[q])] = v;
        l[v] = q; p[v] = m; t[p[m]][toi(a[l[m])]] = m;
        v = s[p[m]]; q = l[m];
        while(q < r[m]) v = t[v][toi(a[q])], q += r[v] - l[v];
        s[m] = q == r[m] ? v : m+2;
        q = r[v] - (q - r[m]); m += 2;
        goto suff;
    }
}
SuffixTree(string a) : a(a), t(2*sz(a)+5), l(t.size()), r(t.size(), sz(a)),
    p(t.size()), s(t.size()){
    for(auto &x : t) x.fill(-1);
    t[1].fill(0); s[0] = 1;
    l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
    rep(i, 0, sz(a)) ukkadd(i, toi(a[i]));
}
ll distinctSubstrings(){
    ll ans = 0;
    rep(i, 2, m) ans += r[i] - l[i];
    return ans;
}
pii best;
int lcs(int node, int i1, int i2, int len){
    if(l[node] <= i1 && i1 < r[node]) return 1;
    if(l[node] <= i2 && i2 < r[node]) return 2;
    int mask = 0;
    if(node) len += r[node] - l[node];
    rep(c, 0, ALPHA) if(t[node][c] != -1){
        mask |= lcs(t[node][c], i1, i2, len);
    }
    if(mask == 3) best = max(best, {len, r[node] - len});
    return mask;
}
static pii LCS(string s, string t){
    SuffixTree st(s+'{'+t+'}');
    st.lcs(0, sz(s), sz(s)+1+sz(t), 0);
    return st.best;
}
};

```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(—, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.
Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

f35677, 66 lines

```

struct AhoCorasick {
    enum { alpha = 26, first = 'A' }; // change this!
    struct Node {
        // (nmatches is optional)
        int back, next[alpha], start = -1, end = -1, nmatches = 0;
        Node(int v){ memset(next, v, sizeof(next)); }
    };
    vector<Node> N;
    vi backp;
    void insert(string& s, int j){
        assert(!s.empty());
        int n = 0;
        for(char c : s){
            int& m = N[n].next[c - first];

```

```

            if(m == -1){ n = m = sz(N); N.emplace_back(-1); }
            else n = m;
        }
        if(N[n].end == -1) N[n].start = j;
        backp.push_back(N[n].end);
        N[n].end = j;
        N[n].nmatches++;
    }
    AhoCorasick(vector<string>& pat) : N(1, -1){
        rep(i, 0, sz(pat)) insert(pat[i], i);
        N[0].back = sz(N);
        N.emplace_back(0);

        queue<int> q;
        for(q.push(0); !q.empty(); q.pop()){
            int n = q.front(), prev = N[n].back;
            rep(i, 0, alpha){
                int &ed = N[n].next[i], y = N[prev].next[i];
                if(ed == -1) ed = y;
                else {
                    N[ed].back = y;
                    (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
                        = N[y].end;
                    N[ed].nmatches += N[y].nmatches;
                    q.push(ed);
                }
            }
        }
        vi find(string word){
            int n = 0;
            vi res; // ll count = 0;
            for(char c : word){
                n = N[n].next[c - first];
                res.push_back(N[n].end);
                // count += N[n].nmatches;
            }
            return res;
        }
        vector<vi> findAll(vector<string>& pat, string word){
            vi r = find(word);
            vector<vi> res(sz(word));
            rep(i, 0, sz(word)){
                int ind = r[i];
                while(ind != -1){
                    res[i - sz(pat[ind]) + 1].push_back(ind);
                    ind = backp[ind];
                }
            }
            return res;
        }
    };
}

```

Sam.h

Description: Generalized suffix automaton for lowercase strings. Construct with total length, insert strings, build, then solve for cnt.
Time: $\mathcal{O}(N\text{alphabet})$

5cc30c, 38 lines

```

struct exSAM{
    static const int C = 27;
    vector<int>len, link, ord, cnt;
    vector<array<int, C>>go;
    int last;
    exSAM(int n){
        len.reserve(4 * n + 5), link.reserve(4 * n + 5), cnt.
            reserve(4 * n + 5), go.reserve(4 * n + 5);
        node(); link[0] = -1, last = 0;
    }
    int node(){

```

```
len.pb(0), link.pb(0), cnt.pb(0), go.pb({}); return sz(len) - 1;
}
void extend(int c){
    int cur = node(), p = last; len[cur] = len[last] + 1, cnt[cur] = 1, last = cur;
    for(; p != -1 and !go[p][c]; p = link[p]) go[p][c] = cur;
    if(p == -1) return link[cur] = 0, void();
    int q = go[p][c];
    if(len[p] + 1 == len[q]) return link[cur] = q, void();
    int cl = node(); go[cl] = go[q], len[cl] = len[p] + 1, link[cl] = link[q];
    for(; p != -1 and go[p][c] == q; p = link[p]) go[p][c] = cl;
    link[q] = link[cur] = cl;
}
void insert(const string &s){
    for(char c : s) extend(c - 'a');
    extend(26);
}
void build(){
    int n = sz(len); ord.resize(n); vector<int>sum(len[last] + 1);
    rep(i, 0, n) ++sum[len[i]];
    partial_sum(all(sum), sum.begin());
    rep(i, 0, n) ord[--sum[len[i]]] = i;
}
void solve(){
    for(int i = sz(ord) - 1; i; --i){
        int u = ord[i]; cnt[link[u]] += cnt[u];
    }
}
};
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).
Time: $\mathcal{O}(\log N)$

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R){
    if(L == R) return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while(it != is.end() && it->first <= R){
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if(it != is.begin() && (--it)->second >= L){
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.insert(before, {L, R});
}

void removeInterval(set<pii>& is, int L, int R){
    if(L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if(it->first == L) is.erase(it);
    else (int&)it->second = L;
    if(R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add || R.empty(). Returns empty set on failure (or if G is empty).
Time: $\mathcal{O}(N \log N)$

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I){
    vi S(sz(I)), R;
    iota(all(S), 0);
    sort(all(S), [&](int a, int b){ return I[a] < I[b]; });
    T cur = G.first;
    int at = 0;
    while(cur < G.second){ // (A)
        pair<T, int> mx = make_pair(cur, -1);
        while(at < sz(I) && I[S[at]].first <= cur){
            mx = max(mx, make_pair(I[S[at]].second, S[at]));
            at++;
        }
        if(mx.second == -1) return {};
        cur = mx.first;
        R.push_back(mx.second);
    }
    return R;
}
```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.
Usage: constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});
Time: $\mathcal{O}(k \log \frac{n}{k})$

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q){
    if(p == q) return;
    if(from == to){
        g(i, to, p);
        i = to; p = q;
    }else{
        int mid = (from + to) >> 1;
        rec(from, mid, f, g, i, p, f(mid));
        rec(mid+1, to, f, g, i, p, q);
    }
}

template<class F, class G>
void constantIntervals(int from, int to, F f, G g){
    if(to <= from) return;
    int i = from; auto p = f(i), q = f(to-1);
    rec(from, to-1, f, g, i, p, q);
    g(i, to, q);
}
```

10.2 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in [a, b] that maximizes f(i), assuming that f(a) < ... < f(i) ≥ ... ≥ f(b). To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f, change it to >, also at (B).
Usage: int ind = ternSearch(0, n-1, [&](int i){return a[i];});
Time: $\mathcal{O}(\log(b - a))$

```
template<class F>
int ternSearch(int a, int b, F f){
    assert(a <= b);
    while(b - a >= 5){
        int mid = (a + b) / 2;
```

```
        if(f(mid) < f(mid+1)) a = mid; // (A)
        else b = mid+1;
    }
    rep(i, a+1, b+1) if(f(a) < f(i)) a = i; // (B)
    return a;
}
```

LIS.h

Description: Compute indices for the longest increasing subsequence.
Time: $\mathcal{O}(N \log N)$

```
template<class I> vi lis(const vector<I>& S){
    if(S.empty()) return {};
    vi prev(sz(S));
    typedef pair<I, int> p;
    vector<p> res;
    rep(i, 0, sz(S)){
        // change 0 -> i for longest non-decreasing subsequence
        auto it = lower_bound(all(res), p{S[i], 0});
        if(it == res.end()) res.emplace_back(), it = res.end()-1;
        *it = {S[i], i};
        prev[i] = it == res.begin() ? 0 : (it-1)->second;
    }
    int L = sz(res), cur = res.back().second;
    vi ans(L);
    while(L--) ans[L] = cur, cur = prev[cur];
    return ans;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.
Time: $\mathcal{O}(N \max(w_i))$

```
int knapsack(vi w, int t){
    int a = 0, b = 0, x;
    while(b < sz(w) && a + w[b] <= t) a += w[b++];
    if(b == sz(w)) return a;
    int m = *max_element(all(w));
    vi u, v(2*m, -1);
    v[a+m-t] = b;
    rep(i, b, sz(w)){
        u = v;
        rep(x, 0, m) v[x+w[i]] = max(v[x+w[i]], u[x]);
        for(x = 2*m; --x > m;) rep(j, max<int>(0, u[x]), v[x])
            v[x-w[j]] = max(v[x-w[j]], j);
    }
    for(a = t; v[a+m-t] < 0; a--);
    return a;
}
```

SimulatedAnnealing.h

```
double factor = 100000;
const int base = 1e9; // remember to run ~ 10 times
for (int it = 1; it <= 1000000; ++it) {
    // ans: answer, nw: current value, rnd(): mt19937 rnd()
    if (exp(-(nw - ans) / factor) >= (double)(rnd() % base) / base)
        ans = nw;
    factor *= 0.99995;
}
```

10.3 Dynamic programming

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$, where the (minimal) optimal k increases with both i and j , one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j - 1]$ and $p[i + 1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b, c) \leq f(a, d)$ and $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.

Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h

Description: $dp[i] = \min_{k \leq i} cost(i, k)$, assuming $opt[i]$ is monotone.

Time: $\mathcal{O}((N + K) \log N)$ cost calls

```
template<class F>
pair<vector<ll>, vi> dcDP(int n, int k, F cost){
    vector<ll>dp(n, LLONG_MAX);
    vi opt(n);
    auto go = [&](auto self, int l, int r, int ql, int qr)->void{
        if(l >= r) return;
        int m = (l + r) / 2, hi = min(qr, m + 1), who = ql;
        rep(j, ql, hi){
            ll v = cost(m, j);
            if(v < dp[m]) dp[m] = v, who = j;
        }
        opt[m] = who;
        self(self, l, m, ql, who + 1);
        self(self, m + 1, r, who, qr);
    };
    go(go, 0, n, 0, k);
    return {dp, opt};
}
```

10.4 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });`
converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

10.5.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.
- `rep(b,0,K) rep(i,0,(1 << K))`
if `(i & 1 << b) D[i] += D[i^(1 << b)];`
computes all sums of subsets.

10.5.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.

- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a \% b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to $a \pmod b$ in the range $[0, 2b)$.

```
typedef unsigned long long ull;
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m(-1ULL / b){}
    ull reduce(ull a){ // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

```
// Either globally or in a single class:
alignas(max_align_t) static char buf[450 << 20];
static size_t bumpPos = sizeof buf;
void* operator new(size_t s){
    assert(s <= bumpPos);
    bumpPos = (bumpPos-s) & ~(alignof(max_align_t)-1);
    return (void*)&buf[bumpPos];
}
void operator delete(void*) noexcept {}
void operator delete(void*, size_t) noexcept {}
size_t bumpMark(){ return bumpPos; }
void bumpReset(size_t p){ assert(bumpPos <= p && p <= sizeof buf); bumpPos = p; }
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

"BumpAllocator.h"

```
template<class T> struct ptr {
    unsigned ind;
    ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0){
        assert(ind < sizeof buf);
    }
    T& operator*() const { return *(T*)(buf + ind); }
    T* operator->() const { return &*this; }
    T& operator[](int a) const { return (&*this)[a]; }
    explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.

Usage: `vector<vector<int, small<int>>> ed(N);`

```
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template<class T> struct small {
    typedef T value_type;
    small(){}
    template<class U> small(const U&){}
    T* allocate(size_t n){
        buf_ind -= n * sizeof(T);
        buf_ind &= 0 - alignof(T);
        return (T*)(buf + buf_ind);
    }
    void deallocate(T*, size_t){}
};
```

Unrolling.h

Description: Calls `f(i)` for `i` in `[l, r)`, four iterations at a time.

```
template<class F> void unroll4(int l, int r, F f){
    while((l&3) and l<r) f(l++);
    for(;l+4<=r;l+=4) f(l), f(l+1), f(l+2), f(l+3);
    while(l<r) f(l++);
}
```